
# Enterprise Agent Capability Implementation

## Skill × Harness × Governance

ICSOFT Intelligent Computing and Software Center
浙江大学 智能计算与软件中心
www.icsoft.zju.edu.cn

# Outline

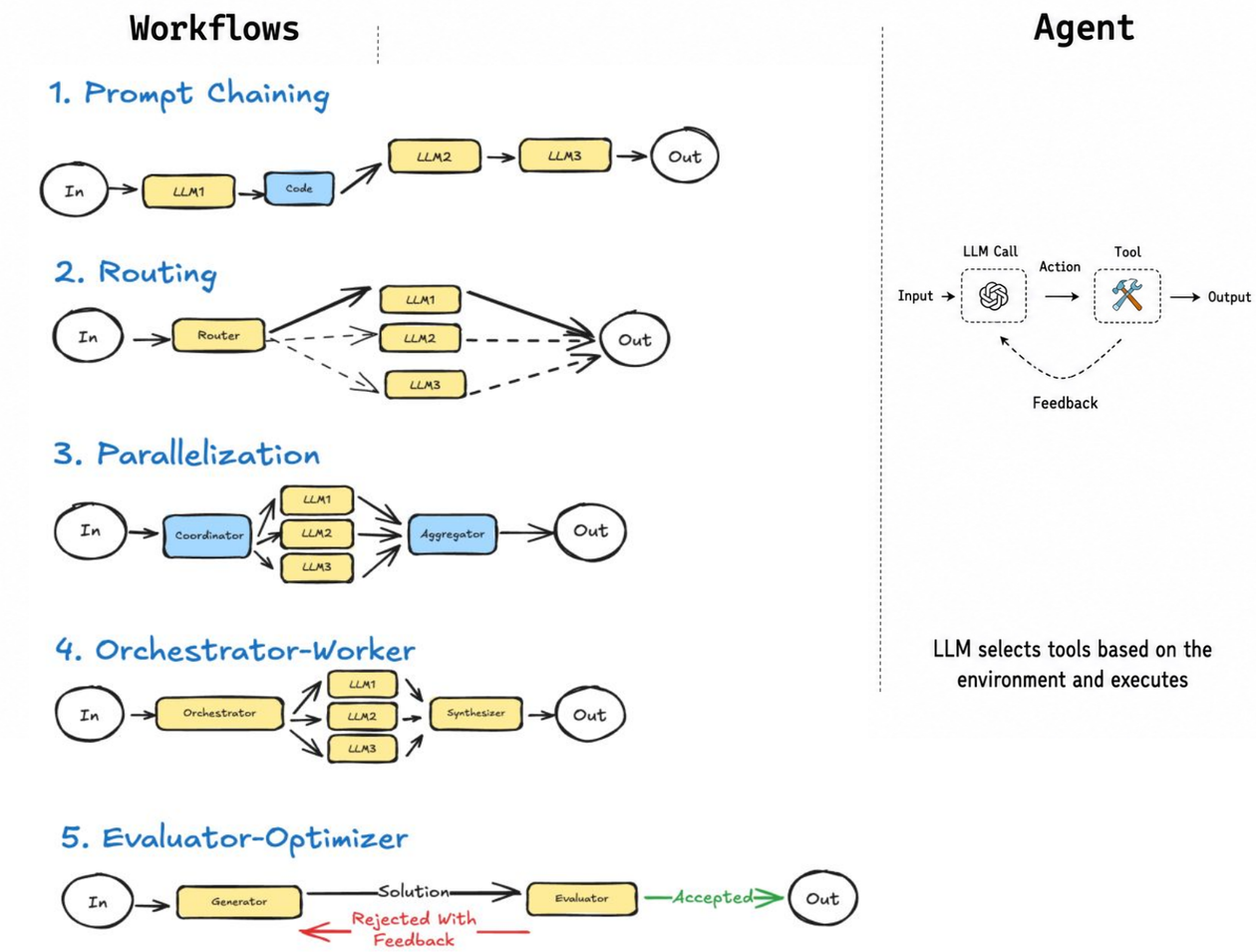
1. Why Enterprise Agent Needs More Than LLM
2. Skill: Capability Packaging
3. Harness: Controlled Execution
4. Loop Engineering: Completeness Closure
5. Governance & Roadmap

# 1. Why Enterprise Agent Needs More Than LLM

# Why Enterprises NeedAI Agent?

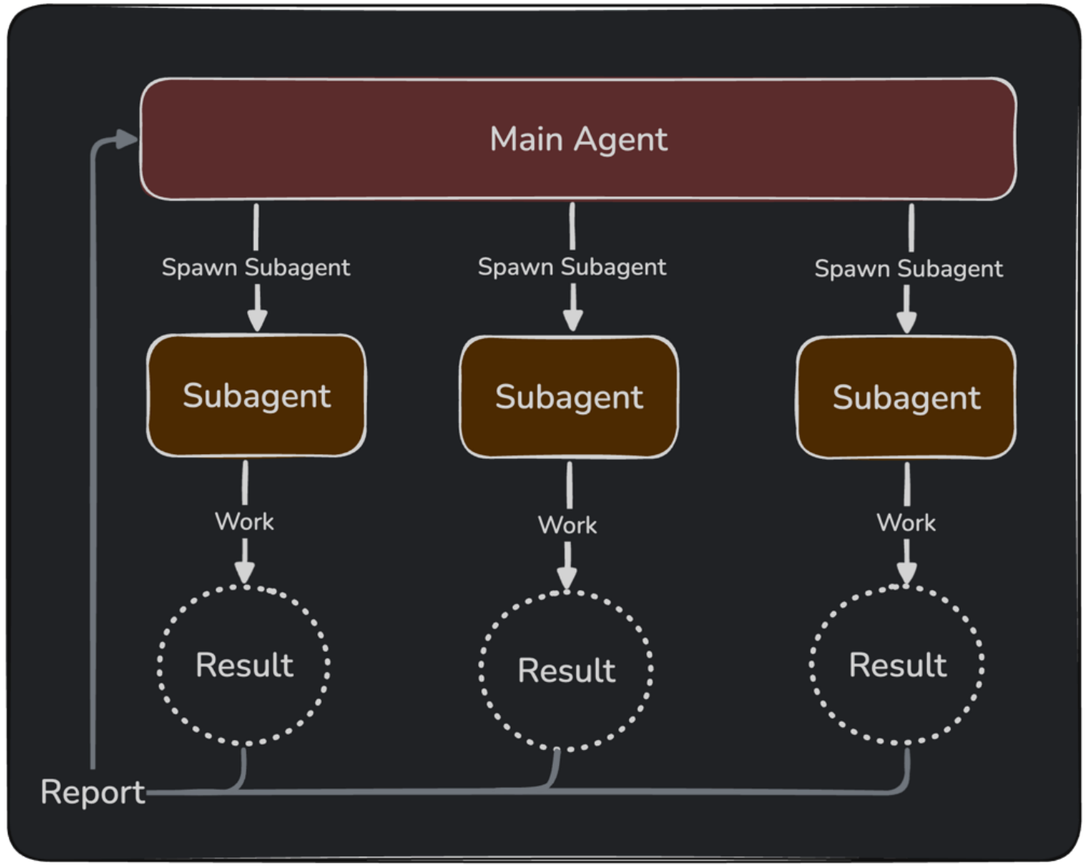
When a task cannot be fully predefined and requires dynamic planning plus interaction with people or systems, Agent the Agent framework becomes a natural choice

- Uncertainty
- Dynamic Planning
- Interactive Decision-making

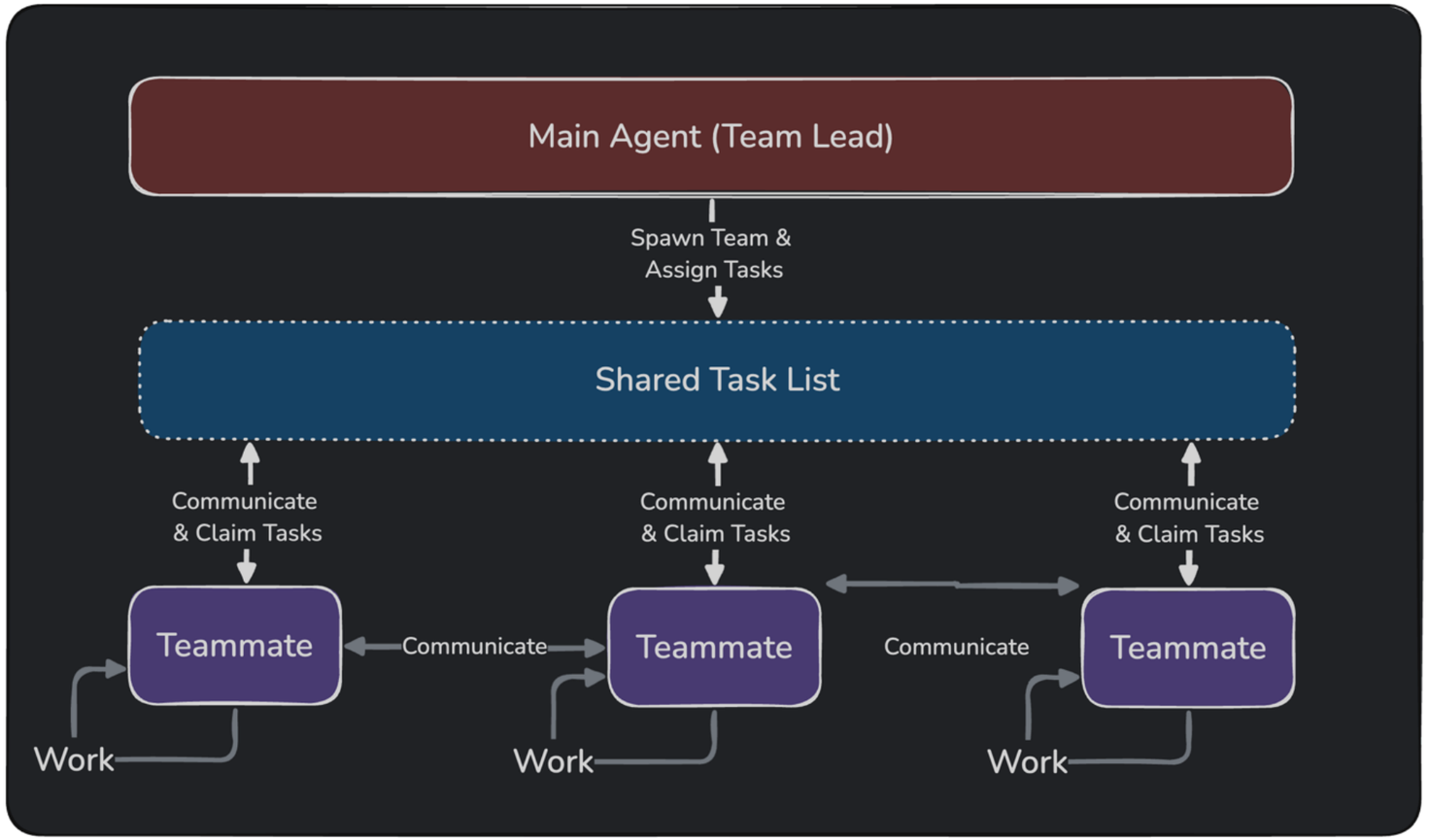


# Why Enterprises NeedAI Agent?

Subagents




Agent Teams



## ReAct Tool-Call Agent

- Tool tells the LLM what it can do actions
- Role: call external systems or interfaces
- Strengths： strong expansion capability
- Limitations： raw tool calls can easily get out of control

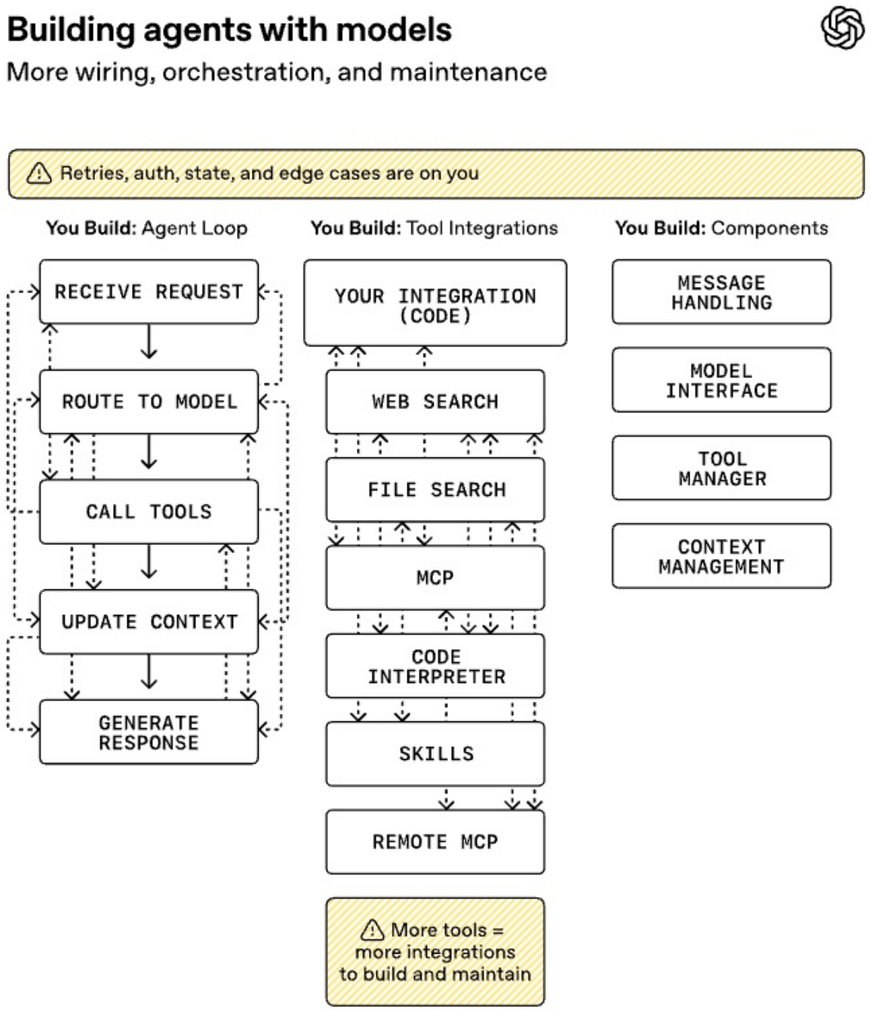
## Workflow

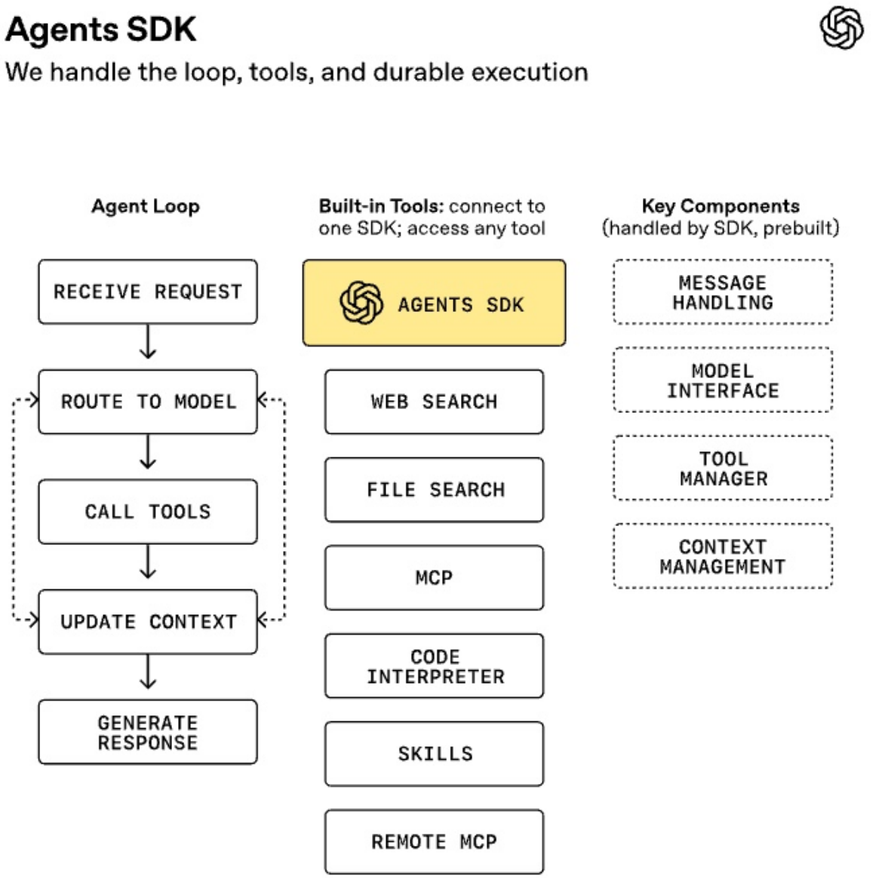
- Workflow turns tasks into fixed processes
- Role： breaks tasks into stable business processes
- Strengths： stable, controllable, and auditable
- Limitations： too rigid and difficult to adapt to complex tasks.

# When to use an Agent

- The steps cannot be exhaustively enumerated.
- Plans need to be adjusted based on intermediate results.
- Results need to be verified during execution.
- Multiple recovery paths exist after failure.

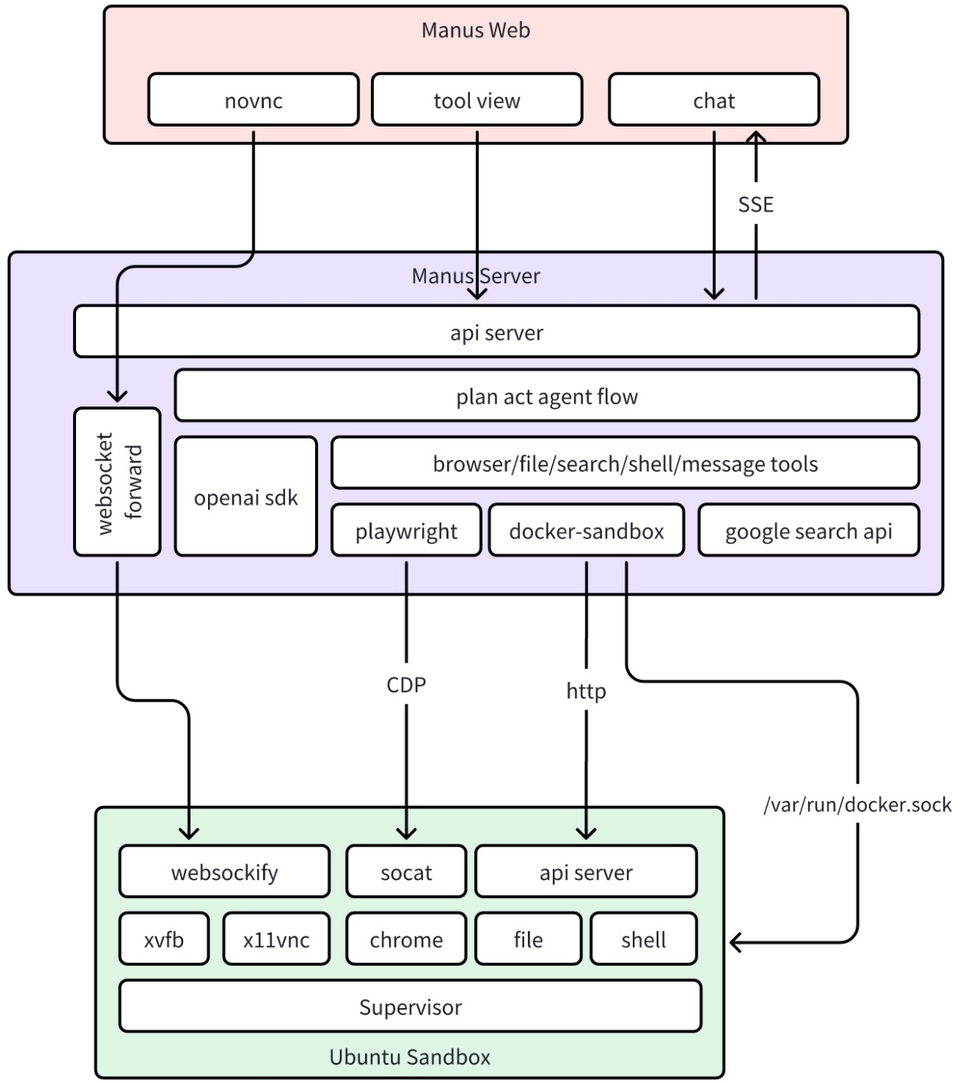
Without SDK building an Agent vs using Agents SDK to build an Agent —the differences.



## Sandbox - CLI

- File: http api
- Shell: http api
- Browser: playwright
  - browser_navigate
  - browser_view
  - browser_click
  - browser_input
  - browser_scroll_down
  - browser_scroll_up
  - browser_press_key
  - browser_select_option
  - browser_console_exec
  - browser_console_view
  - shell_exec
  - shell_view
  - shell_wait
  - shell_write_to_process
  - shell_kill_process
  - file_read
  - file_read_list
  - file_write
  - file_str_replace
  - file_find_in_content
  - file_find_by_name




**Fundamental** **Single Responsibility & Orthogonality** **Security**

# Sandobxing AI agents

- E2B
  - Open-source
  - Memory Shapshots
  - Pause / Resume

| Feature | Daytona | E2B |
|---|---|---|
| **Isolation technology** | Docker containers | Firecracker microVMs |
| **Kernel isolation** | Shared host kernel | Dedicated kernel per session |
| **Session persistence** | Configurable, long-term | Session-based |
| **Primary use case** | Persistent development workspaces | Ephemeral code execution |
| **Deployment model** | Managed (codebase closed-source since June 2026) | Managed service |

- Claudeflare dynamic-workers

```
import { Workspace } from "@cloudflare/shell";
import { stateTools } from "@cloudflare/shell/workers";
import { DynamicWorkerExecutor, resolveProvider } from "@cloudflare/codemode";

const workspace = new Workspace({
  sql: this.ctx.storage.sql, // Works with any DO's SqlStorage, D1, or custom SQL b
  r2: this.env.MY_BUCKET, // large files spill to R2 automatically
  name: () => this.name   // lazy — resolved when needed, not at construction
});

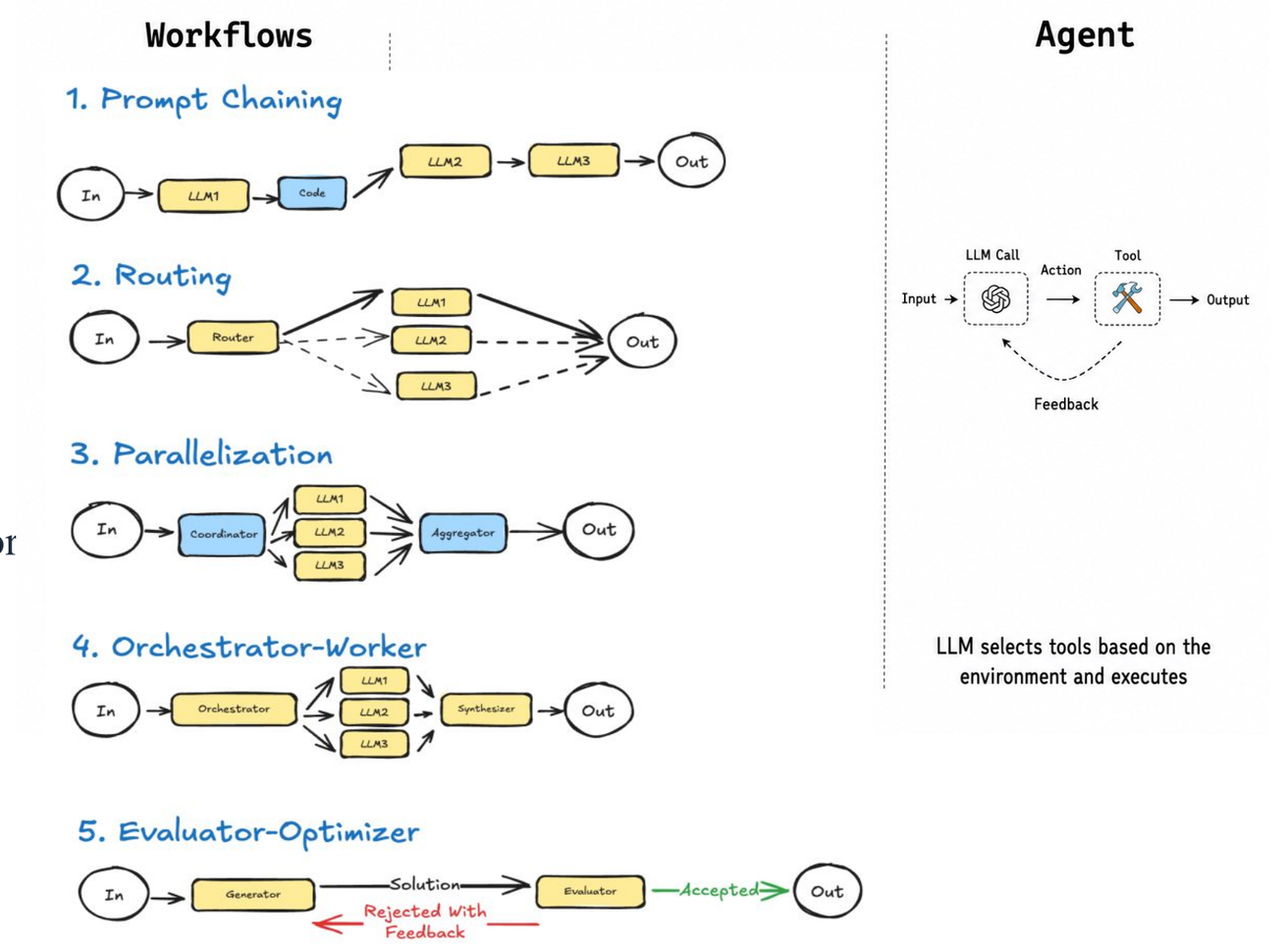
// Code runs in an isolated Worker sandbox with no network access
const executor = new DynamicWorkerExecutor({ loader: env.LOADER });

// The LLM writes this code; `state.*` calls dispatch back to the host via RPC
const result = await executor.execute(
  `async () => {
    // Search across all TypeScript files for a pattern
    const hits = await state.searchFiles("src/**/*.ts", "answer");
    // Plan multiple edits as a single transaction
    const plan = await state.planEdits([
      { kind: "replace", path: "/src/app.ts",
        search: "42", replacement: "43" },
      { kind: "writeJson", path: "/src/config.json",
        value: { version: 2 } }
    ]);
    // Apply atomically — rolls back on failure
    return await state.applyEditPlan(plan);
  }`,
  [resolveProvider(stateTools(workspace))]
);
```

# Why “just connecting an LLM” cannot meet enterprise implementation needs

## Clever vs. Reliability

- Enterprise Execution Scope
  - Access authorization
  - API call
  - Fallback, Roll-back on failure
  - Human-in-loop
- Reusability
- Auditable
- Verifiable Evidence
  - Avoid Fake Completeness
- Authority Management
  - e.g., agent kills sandbox supervisor



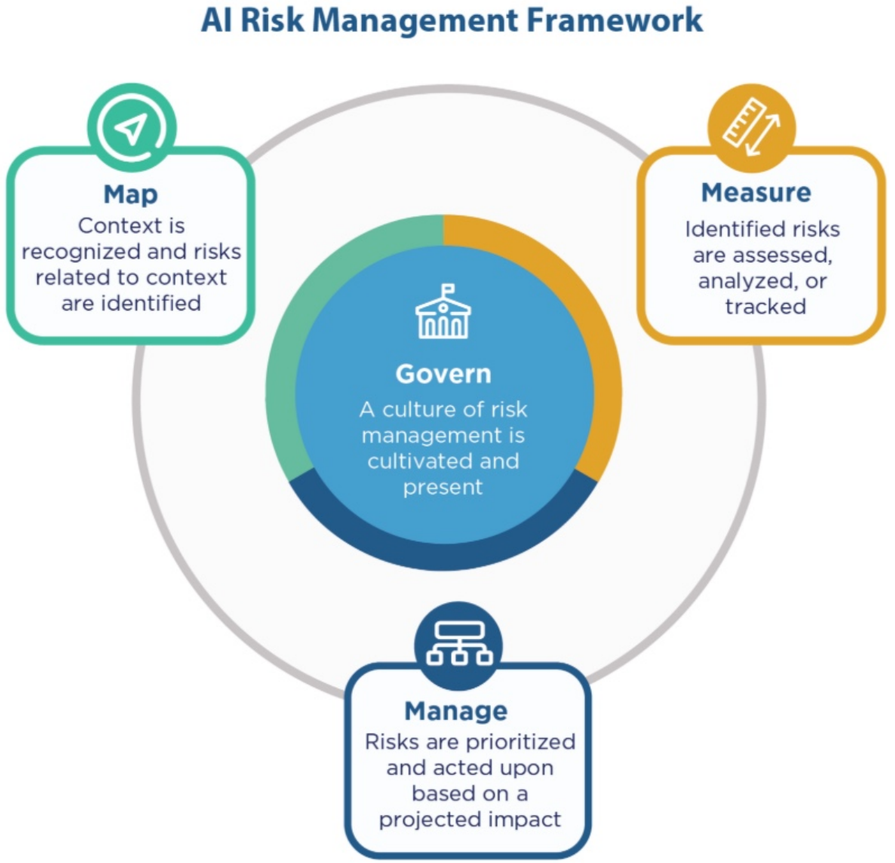
# Capability Layers Enterprises Really Need

Enterprise Agent
- Model
- Instructions
- Skills
- Context and Knowledge
- Tools
- Execution Loop
- State
- Verification
- Permissions and Approval
- Audit and Observability
- Governance

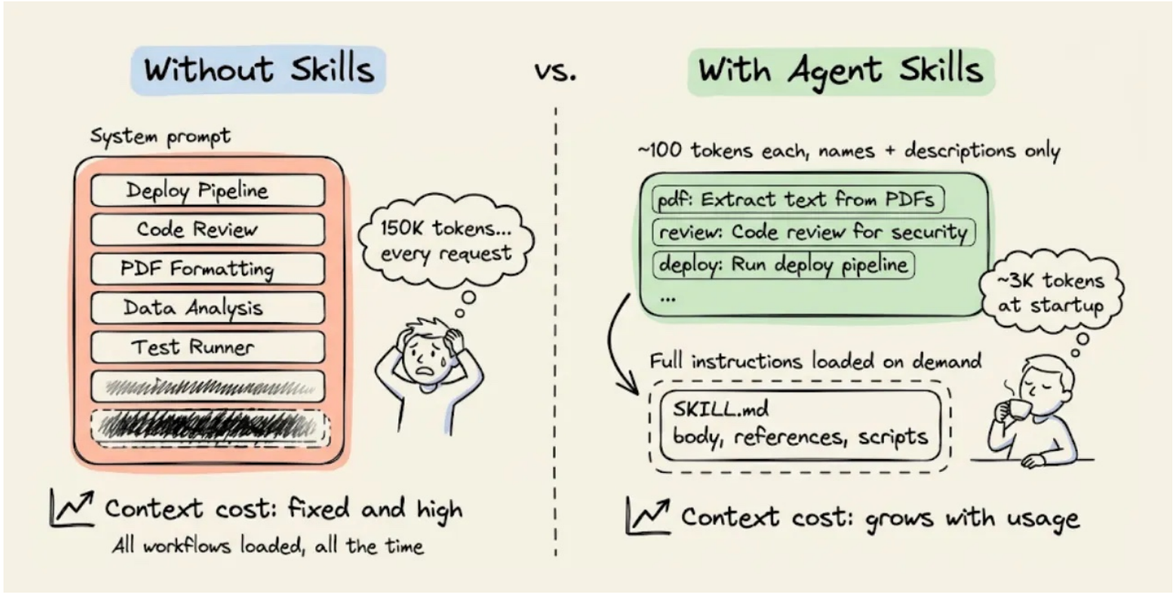
**More powerful models can raise the upper bound of reasoning performance, yet they cannot automatically resolve issues concerning permissions, auditing, rollbacks, data isolation and business accountability.**

# Capability Layers Enterprises Really Need

- Fundamental Coding Capabilities -- Shell Cli
- Controlled Atomic Execution Capabilities – Tools (MCP)
- Reusable Capabilities -- Skill
- Controlled Execution Environment -- Harness
- Continue Execution and Self-Repairing – Loop Engineering
- Organization-level Governance System -- Governance

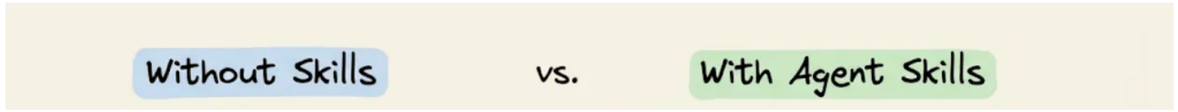


# 2. Skill: Capability Packaging


Read before execution

# 2. Skill: Capability Packaging





```
{
 "tools": [
  {
   "function_declarations": [
    {
     "name": "get_current_weather",
     "description": "获取指定城市的实时天气预报信息",
     "parameters": {
      "type": "OBJECT",
      "properties": {
       "location": {
        "type": "STRING",
        "description": "城市名称，例如：北京、上海"
       },
       "unit": {
        "type": "STRING",
        "enum": ["celsius", "fahrenheit"],
        "description": "温度单位，默认为 celsius"
       }
      },
      "required": ["location"]
     }
    },
    {
     "name": "search_flight_tickets",
     "description": "查询两个城市之间的机票航班信息",
     "parameters": {
      "type": "OBJECT",
      "properties": {
       "origin": {
        "type": "STRING",
        "description": "出发城市"
       },
       "destination": {
        "type": "STRING",
        "description": "到达城市"
       },
       "date": {
        "type": "STRING",
        "description": "出发日期，格式为 YYYY-MM-DD"
       }
      },
      "required": ["origin", "destination", "date"]
     }
    }
   ]
  }
 ],
 "contents": [
  {
   "role": "user",
   "parts": [
    {
     "text": "我想查一下明天（2026-07-23）从上海到北京的机票，顺便看看北京明天天气怎么样。"
    }
   ]
  }
 ]
}
```


- name: get_current_weather
- description:获取指定城市的实时天气预报信息.
- spec path: /workspace/skills/pdf_modifier/SKILL.md

# Skill: a reusable capability package for a specific class of tasks.

A skill is a set of instructions - packaged as a simple folder - that teaches Claude how to handle specific tasks or workflows.

A skill is a folder containing:
**SKILL.md** (required): Instructions in Markdown with YAML frontmatter

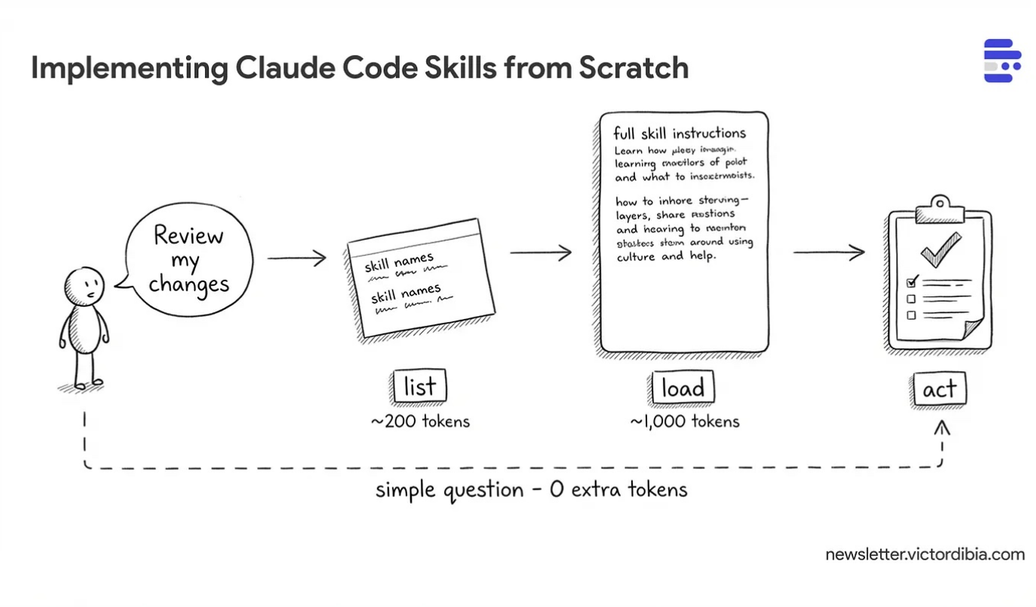
**scripts**/ : Executable code (Python, Bash, etc.) , validate-package,

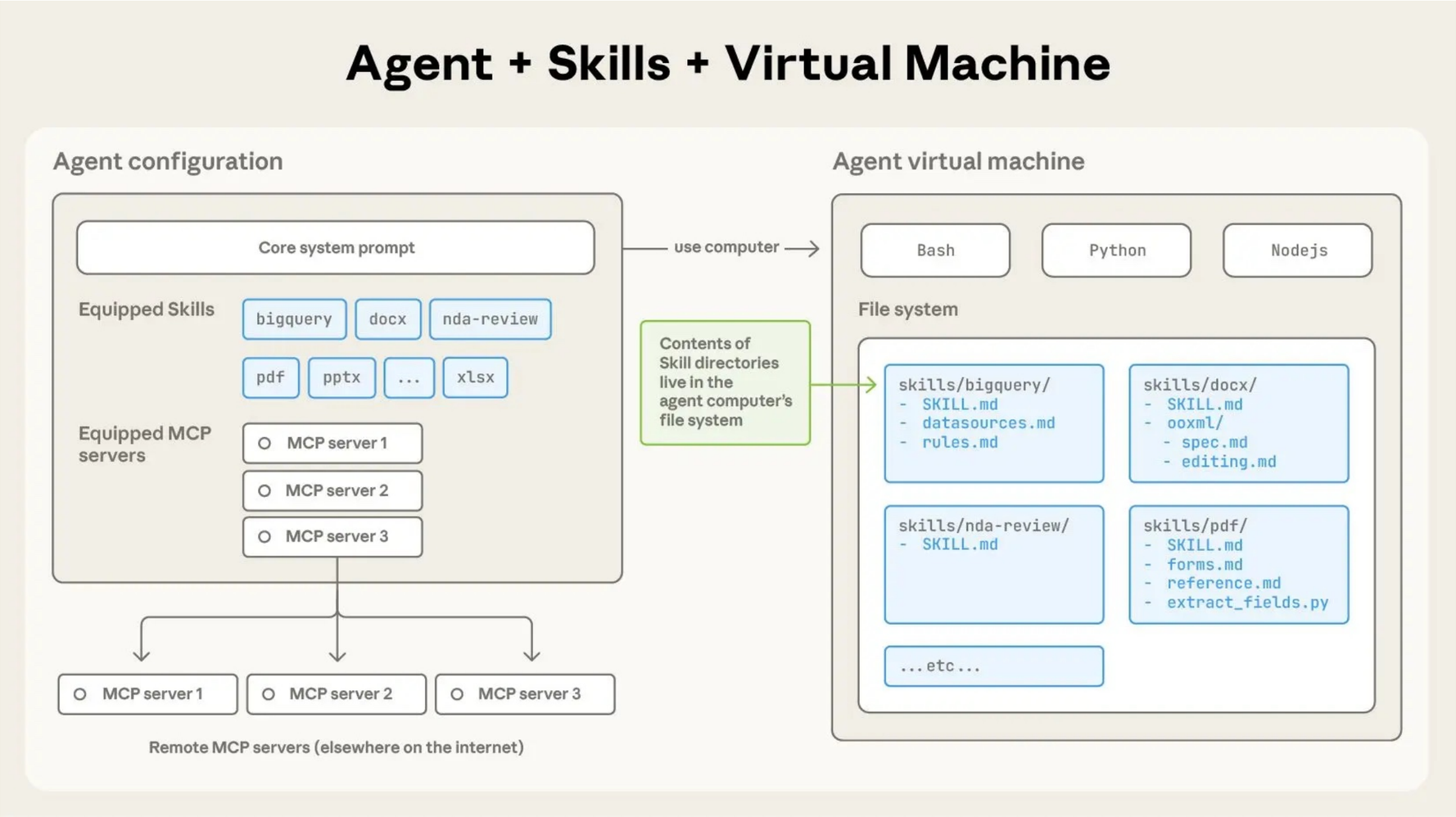
**references**/ required-documents.md, risk-indicators.md, **policy-boundaries.md**

**assets**/ (optional): **Templates**, fonts, icons used in output

**evals**/ (optional): trigger-cases, task-cases, safety-cases

**CHANGELOG.md**




Agent + Skills + Virtual Machine
Agent configuration
Core system prompt
Equipped Skills
bigquery
docx
nda-review
pdf
pptx
...
xlsx
Equipped MCP servers
MCP server 1
MCP server 2
MCP server 3
MCP server 1
MCP server 2
MCP server 3
Remote MCP servers (elsewhere on the internet)
use computer
Contents of Skill directories live in the agent computer's file system
Agent virtual machine
Bash
Python
Nodejs
File system
skills/bigquery/
- SKILL.md
- datasources.md
- rules.md
skills/docx/
- SKILL.md
- ooxml/
- spec.md
- editing.md
skills/nda-review/
- SKILL.md
skills/pdf/
- SKILL.md
- forms.md
- reference.md
- extract_fields.py
...etc...

# How to Designa High-quality Skill

## Core design principles

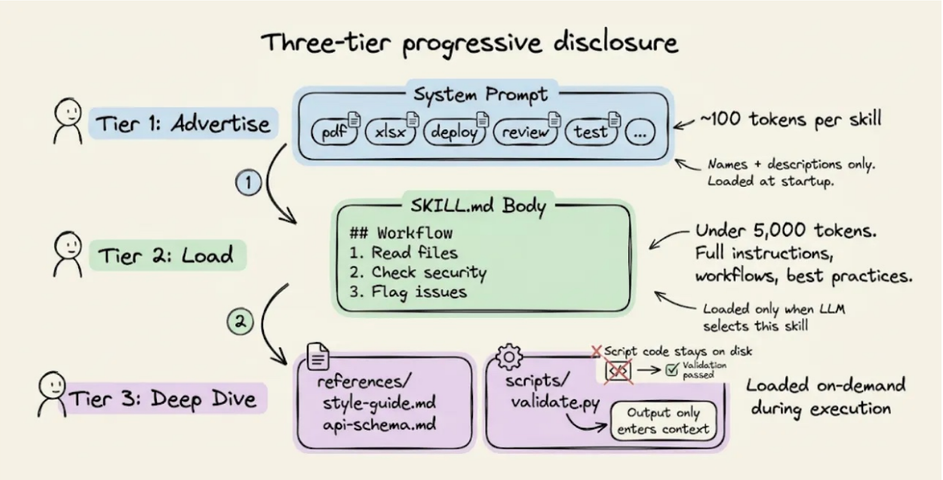
### Progressive Disclosure

Skills use a three-level system:

• First level (YAML frontmatter): Always loaded in Claude's system prompt. Provides just enough information for Claude to know when each skill should be used without loading all of it into context.

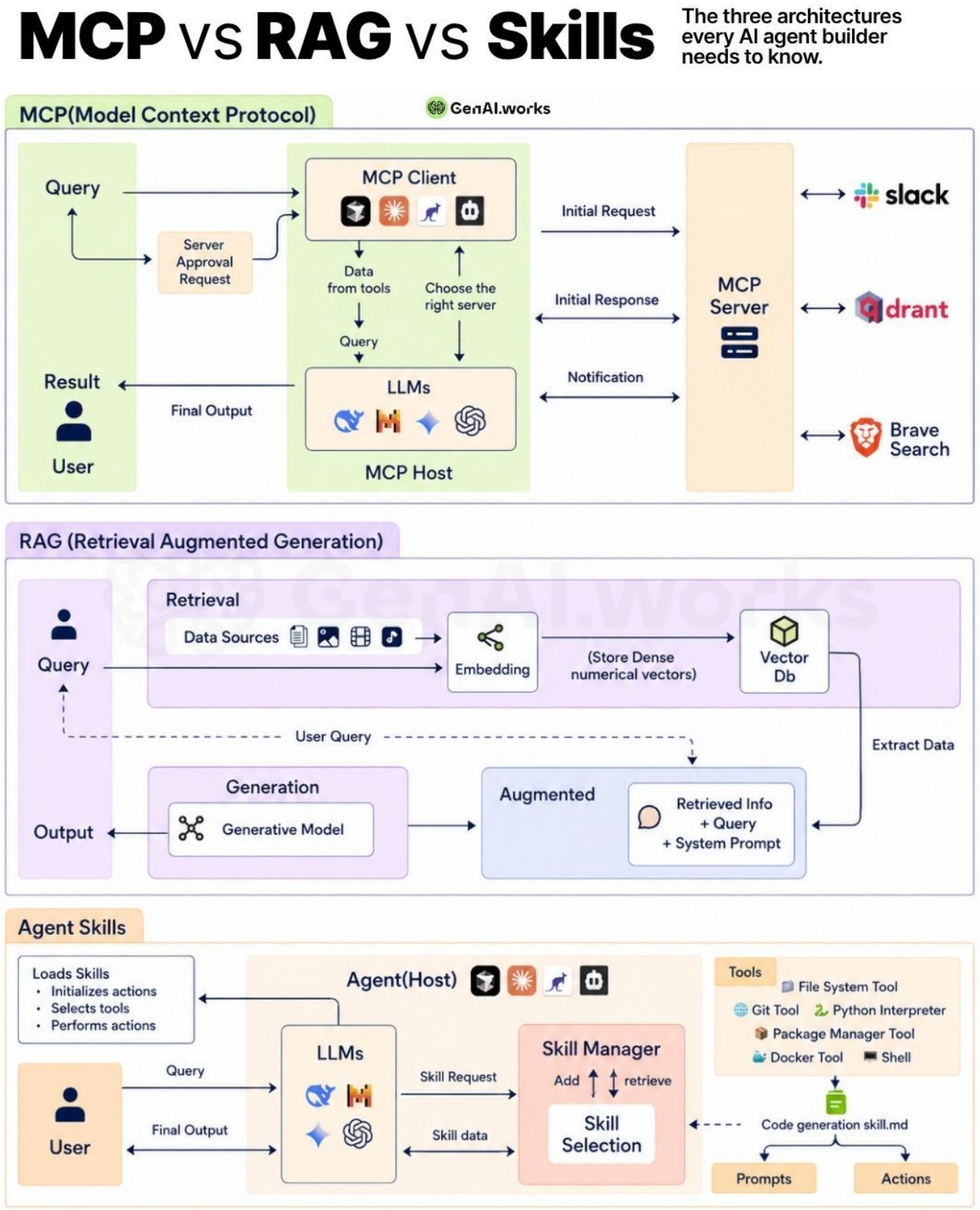
• Second level (SKILL.md body): Loaded when Claude thinks the skill is relevant to the current task. Contains the full instructions and guidance.

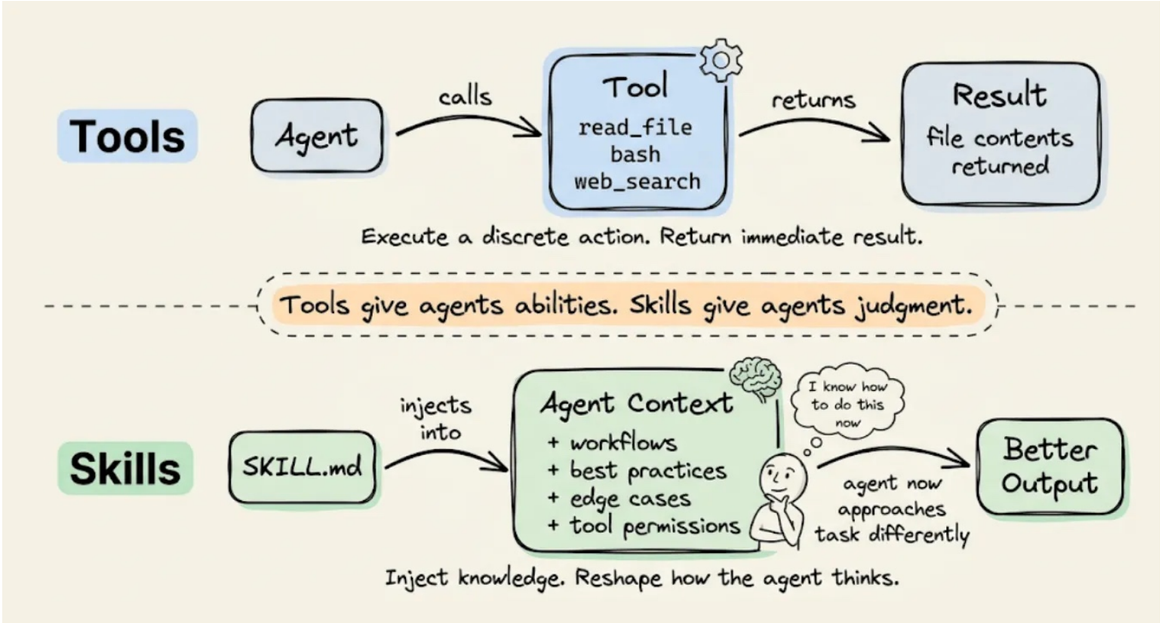
• Third level (Linked files): Additional files bundled within the skill directory that Claude can choose to navigate and discover only as needed.
This progressive disclosure minimizes token usage while maintaining specialized expertise.




**Composability**
Claude can load multiple skills simultaneously. Your skill should work well alongside others, not assume it's the only capability available.

**Portability**
Skills work identically across Claude.ai, Claude Code, and API. Create a skill once and it works across all surfaces without modification, provided the environment supports any dependencies the skill requires.




Skills contain cases and best practices

## Relationship Between Skill and Prompt / RAG / Tool / Workflow

- Prompt determines how the model expresses and reasons.
- RAG provides external knowledge that the model does not originally know.
- Tool enables the model to execute external actions.
- Workflow defines the steps and sequence of a task.
- Skill packages them into reusable task capabilities.

| Layer | Its one job | Changes how often |
|---|---|---|
| **Prompt Engineering** | Define *who the model is* — its persona, tone, guardrails, and baseline behavior | Rarely. This is the foundation. |
| **Skills** | Define *how it thinks* for a specific class of task — the reasoning steps, output schema, domain logic | Occasionally, when workflows evolve |
| **RAG** | Provide *what it needs to know* for this specific query — retrieved from your live knowledge base | Every single request |
| **MCP** | Define *what it can do* — the tools, APIs, and systems it can interact with | When your integrations change |

# With CLI & Skills, Do We Still Need Tools?

**Golden Rule:** Use **Skills** when the LLM needs to act like a developer to **"explore, write scripts, and adapt dynamically"**; use **Tools** when the system needs to act like an API Gateway for **"precise, controlled, and reliable interaction"**. They complement rather than replace each other.

**Tools**

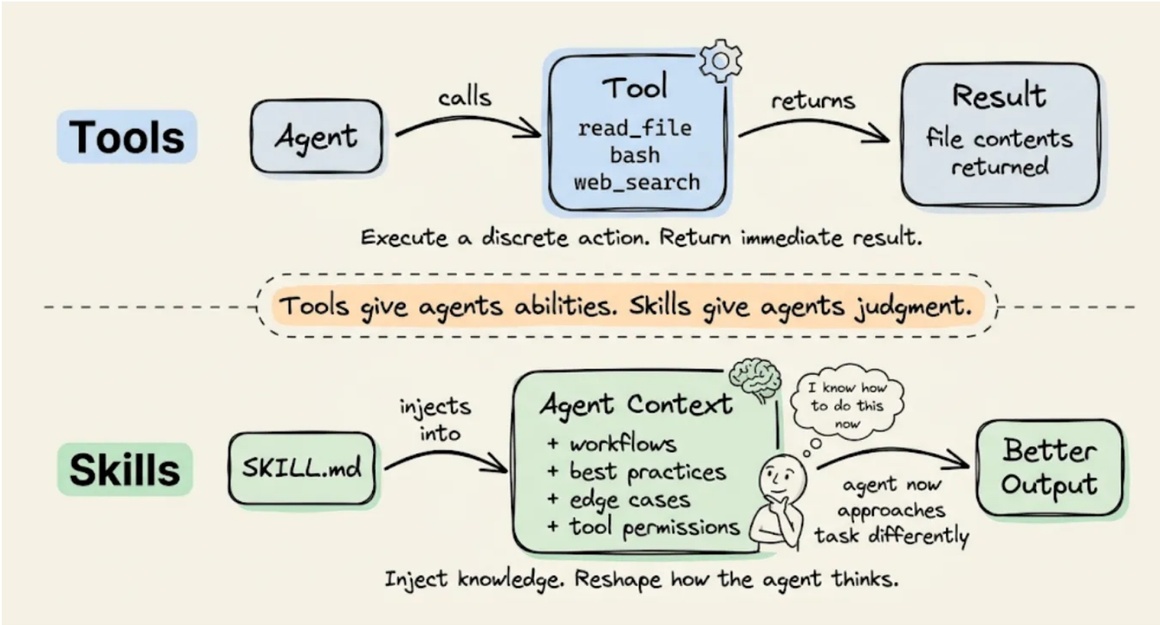
- Json Schema, High Precision & Reliability
- Atomic operation: shell_execution, read_file, browser_navigate
- Reliability and Controllability: deploy_api, check_status
- Enterprise API and Access Control: secret_access, search_orders

**Skills**

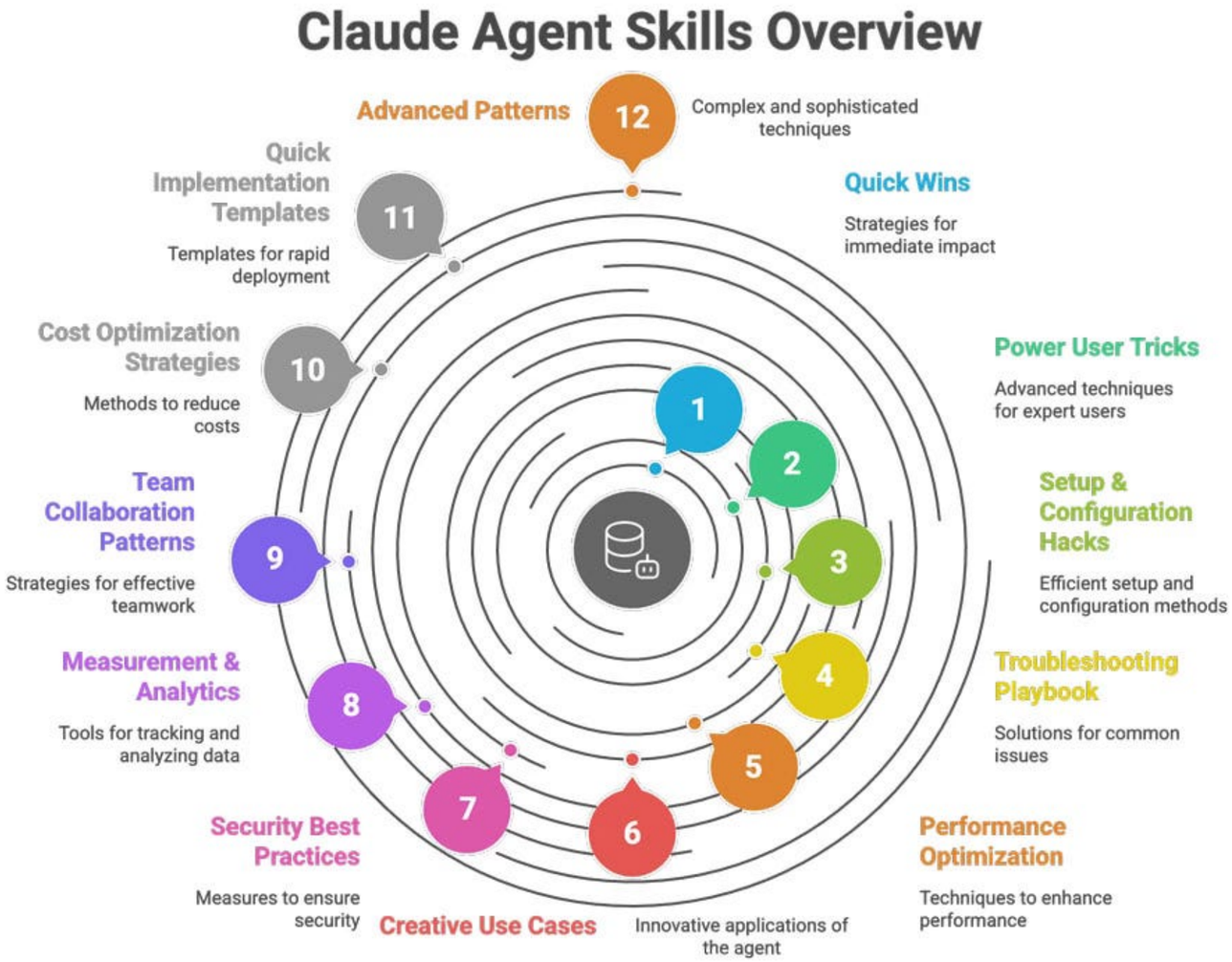
- Versatile, Long Workflows, Flexible
- Reusability with uncertainty: DevOps and Debugging
- Long workflow: media batching: imagemagic + ffmpeg + zip + …

**Prompts**

- General Rules: what can do vs. what must not do
- Tool Trigger and Skill Trigger



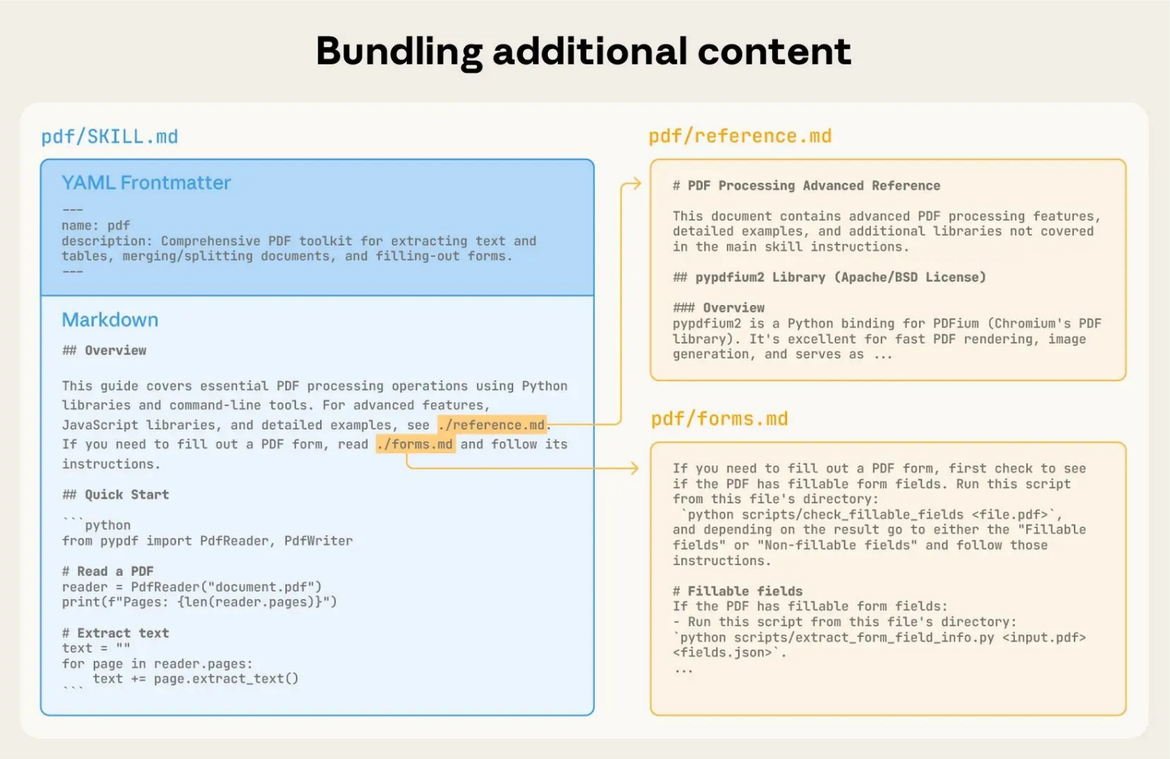
# How to Design a High-quality Skill



# How to Design A High-quality Skill Six Core Principles

A good Skill，is not about “how many features it has,” but whether it is clear enough, stable enough, and easy to trigger correctly.

- Start with 2–3 concrete Use Case use cases
- frontmatter is Skill the most important layer in Skill
- A good Skill，the body must be short, clear, and executable
- Skill should be layered; do not put everything into SKILL.md
- A good Skill，must not only run; it must be triggered correctly
- A good Skill，must be written, tested, and refined iteratively



# Start with 2–3 concrete Use Case use cases

- What does a user want to accomplish?
- What multi-step workflows does this require?
- Which tools are needed (built-in or MCP?)
- What domain knowledge or best practices should be embedded?

Use Case: Loan Application Review

Trigger: User says "review this loan application" or "check loan documents"

Steps:
1. Check required documents
2. Extract company and financial information
3. Identify missing items and risk signals
4. Generate a pre-review summary

Result: Loan document checklist and risk summary

# frontmatter is Skill the most important layer in Skill

The YAML frontmatter is how Claude decides whether to load your skill.

name (required):
• kebab-case only
• No spaces or capitals
• Should match folder name

description(required):
MUST include BOTH:
– What the skill does
– When to use it (trigger conditions)
• Under 1024 characters
• No XML tags (< or >)
• Include specific tasks users might say
• Mention file types if relevant

**Minimal required format**

```
---
name: your-skill-name
description: What it does. Use when user asks to [specific
phrases].
---
```

frontmatter is not supplementary information; it determines whether Skill will be surfaced by the system at the right time.

# A good Skill,  the body must be short, clear, and executable

Skill is not more professional just because it is longer; often the opposite is true—the longer it is, the easier it is to lose control.

- Keep instructions as concise as possible
- Put key rules first
- Use more numbering and bullet point
- Do not cram all complex details into the body

**Recommended structure:**

*Adapt this template for your skill. Replace bracketed sections with your specific content.*

````
---
name: your-skill
description: [...]
---

# Your Skill Name

## Instructions

### Step 1: [First Major Step]
Clear explanation of what happens.

Example:
```bash
python scripts/fetch_data.py --project-id PROJECT_ID
Expected output: [describe what success looks like]
````
````

# Skill should be layered; do not put everything into SKILL.md

Skill should not aim to include as much information as possible, but to provide the right information at the right time. Not all content deserves to stay permanently in the model context.

Do not put to many common sense and step-by-step guidance in Skills

Skill uses the key architecture of Progressive Disclosure

Skills use a three-level system:
- YAML frontmatter
- SKILL.md body
- Linked files

Which information must be placed up front
Which processes must be clearly described
Which details should be deferred until they are truly needed

Skillcan be organized like this：

```
reviewing-loan-applications/
├── SKILL.md
├── reference/
│   ├── required-documents.md
│   ├── risk-flag-rules.md
│   └── policy-checklist.md
├── templates/
│   └── review-memo-template.md
└── scripts/
    └── validate_package.py
```

SKILL.md should only include navigation and the core workflow

**Loan Application Review**

**Workflow**

1. Read the application package.
2. Check document completeness using `reference/required-documents.md`.
3. Identify risk flags using `reference/risk-flag-rules.md`.
4. Produce the review memo using `templates/review-memo-template.md`.
5. Do not make final approval or rejection decisions.

# A good Skill，must not only run; it must be triggered correctly

Skill ’s description is not just explanatory text; it is part of the trigger rules

- Will clearly relevant requests trigger automatically?
- Will it still trigger if the wording changes?
- Will unrelated topics trigger it incorrectly?

```
description: Helps with banking documents.
```

```
description: Reviews retail loan application packages for missing documents, borrower information inconsistencies, and checklist compliance. Use when the user asks to inspect loan files, borrower documents, credit application packages, or application completeness.
```

You can go further and state non-applicable scenarios in the body:

```
## Do not use this Skill for

- Final loan approval or rejection decisions
- Legal advice
- Customer-facing complaint responses
- Investment product recommendations
```

# A good Skill,  must be written, tested, and refined iteratively

It does not treat Skill as something “written once and finished”;
instead, it assumes testing and iteration are required.
Skill issues only appear during real use
Writing Skill is essentially more like:

- Define a working version first
- Test it with real tasks
- Check whether it under-triggers or over-triggers
- Check whether instructions are unclear or the scope is too broad
- Then keep refining, rewriting, and splitting it

Failure points in real tasks
↓
Extract reusable rules
↓
Write a minimum viable Skill
↓
Evaluate with 3–5 test cases
↓
Observe false triggers, missed triggers, and output quality
↓
Modify description / workflow / reference / scripts

Example

- What is this skill for?
- Default Standards and Priority Orders
- Rules
- What is the evidence to declare completeness
- How to verify evidence?
- Use Cases
- Risks

# Skill Evaluation and Launch

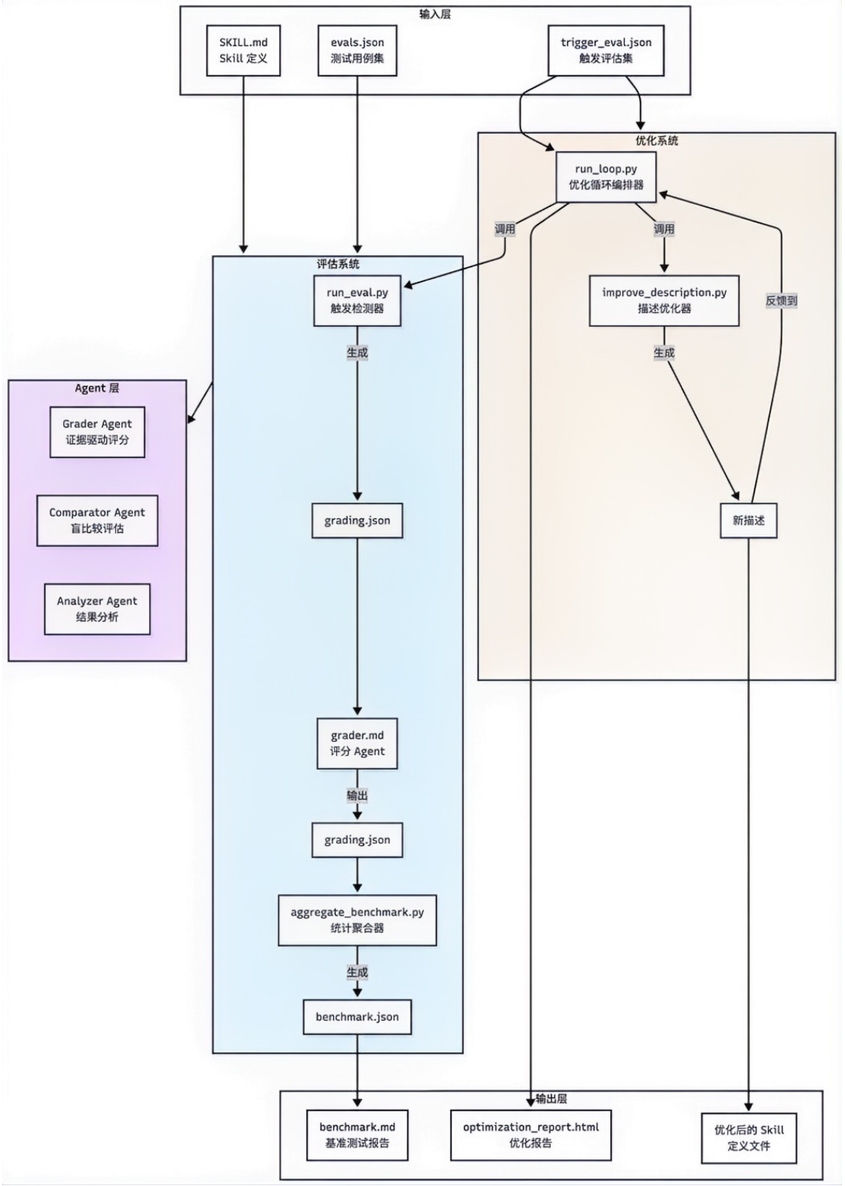
Claude Code official skill-creator plugin includes a complete evaluation system.

Evaluation Metrics

- **Trigger Accuracy**
  - **Positive trigger vs. Negative trigger**
- Output Quality
- **Boundary Compliance**
- Efficiency Metrics

Evaluation

- **Parallel execution**: with-skill and baseline start at the same time
- write assertion
- **Multiple iterations**: improve Skill → rerun → compare iterations → improve again.
- **Record all data**: timing.json（token + latency),grading.json（assertion pass status),feedback.json（human feedback).



# Skill Evaluation and Launch

1. Planning：identify repetitive, high-risk, and strongly process-driven tasks

2. Authoring：create SKILL.md、reference、templates、scripts

3. Security Review：check scripts, external URL、hard-coded credentials, and sensitive-data access

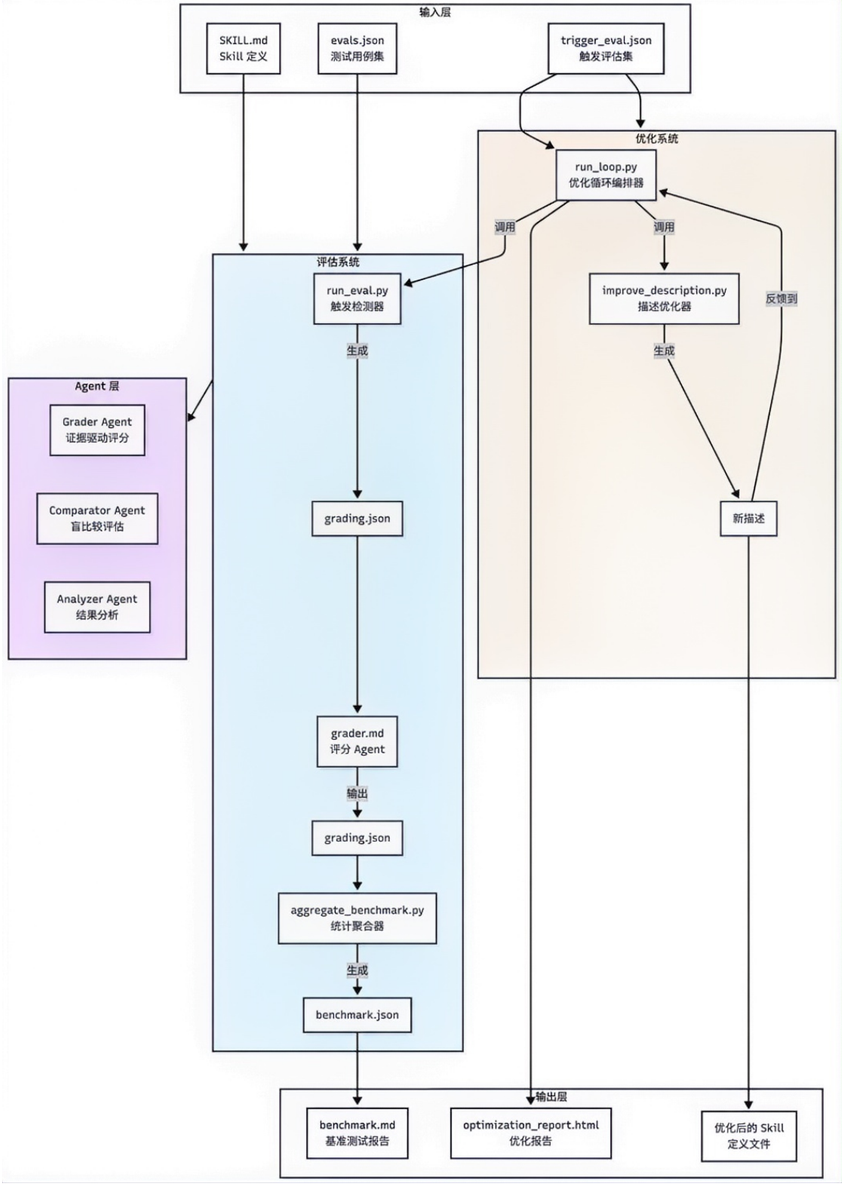
4. Evaluation：trigger accuracy, isolation behavior, coexistence, instruction following, and output quality

5. Canary Release：use by a small group of roles or teams

6. Official Release：upload API / distribute to designated platforms / record version and owner

7. Monitoring and Regression Testing：collect failure cases and rerun evaluations regularly

8. Iterate or Deprecate：each new version must pass the full evaluation before release



# Corporate Loan Pre-Review Skill

## Skill Folder Structure

```
corporate-loan-pre-review/
├── SKILL.md
├── references/
│   ├── required-documents.md
│   ├── risk-indicators.md
│   └── output-template.md
└── scripts/
    └── validate_checklist.py
```

## Frontmatter

```
---
name: corporate-loan-pre-review
description: Assists bank staff in pre-reviewing
corporate loan application materials. Use when the user
asks to review loan documents, check missing items,
or generate a pre-review summary.
---
```

## Use Case：Loan Application Review

Use Case: Loan Application Review

Trigger: User says "review this loan application"
or "check loan documents"

Steps:
1. Check required documents
2. Extract company and financial information
3. Identify missing items and risk signals
4. Generate a pre-review summary

Result: Loan document checklist and risk summary

# Corporate Loan Pre-Review Skill

## Workflow

1. Identify loan context
2. Check required documents
3. Extract key company information
4. Identify preliminary risk signals
5. Generate pre-review summary

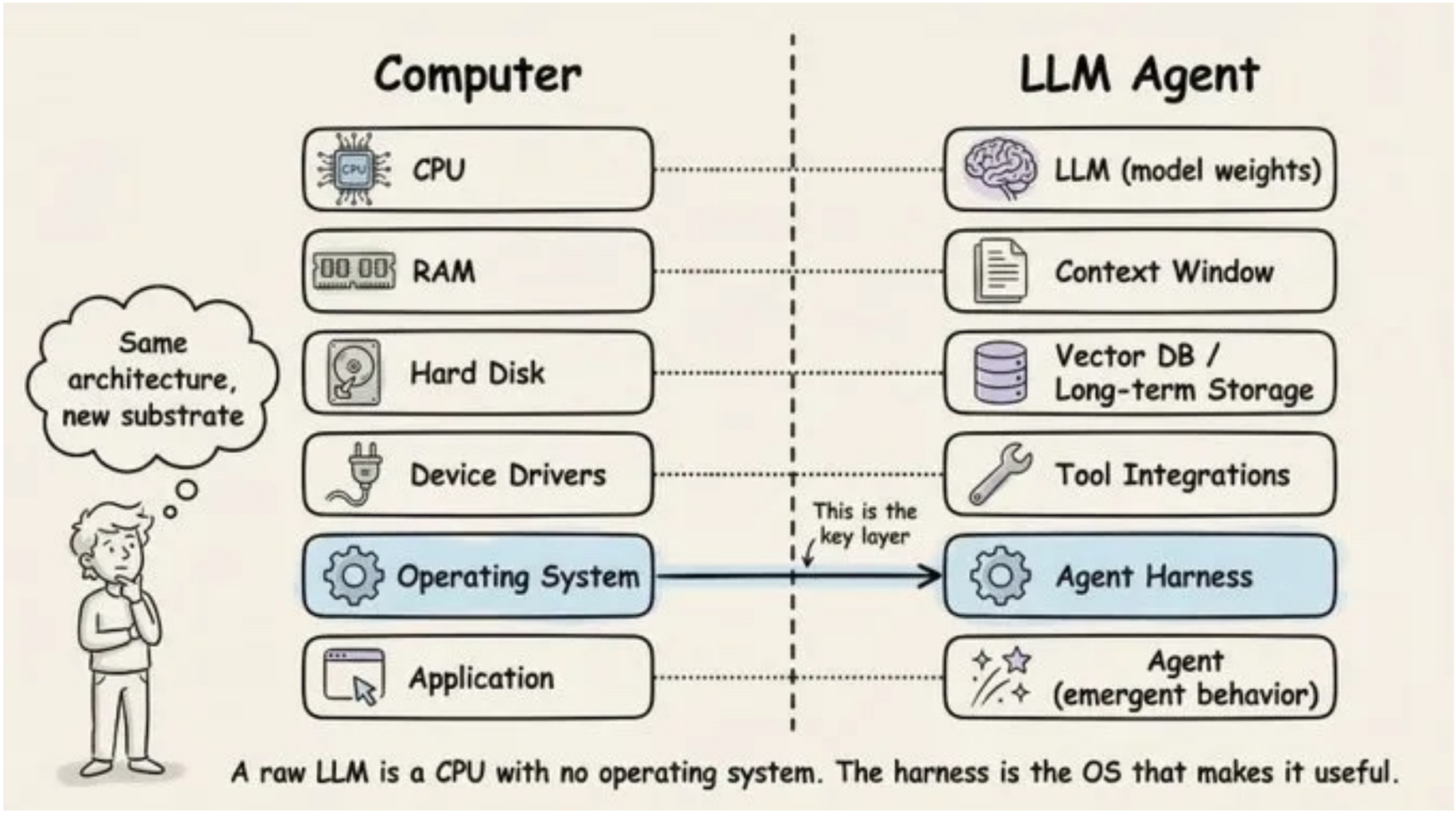
## Output Template

Corporate Loan Pre-Review Summary

1. Basic Information
2. Document Completeness
3. Financial Overview
4. Preliminary Risk Signals
5. Manual Review Items
6. Compliance Note

# 3. Harness: Controlled Execution

# Harness Concept: A Controlled Execution Framework for Agents



# Harness Concept: A Controlled Execution Framework for Agents

What is a harness: Everything in the engineering infrastructure outside the model weights.

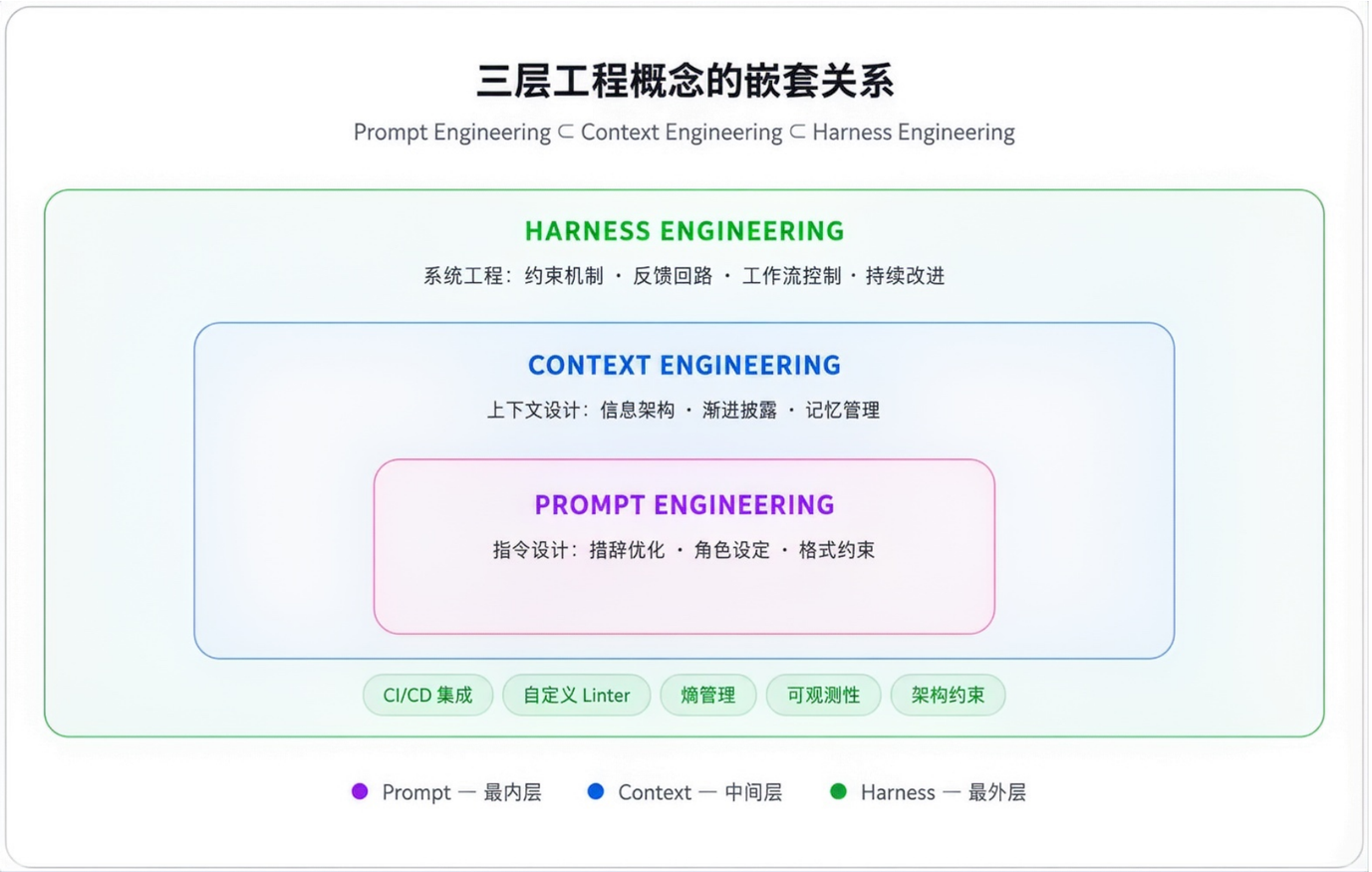
**Clear Scope**. The repo is the single source of truth: Anything the agent cannot see, for all practical purposes, does not exist.

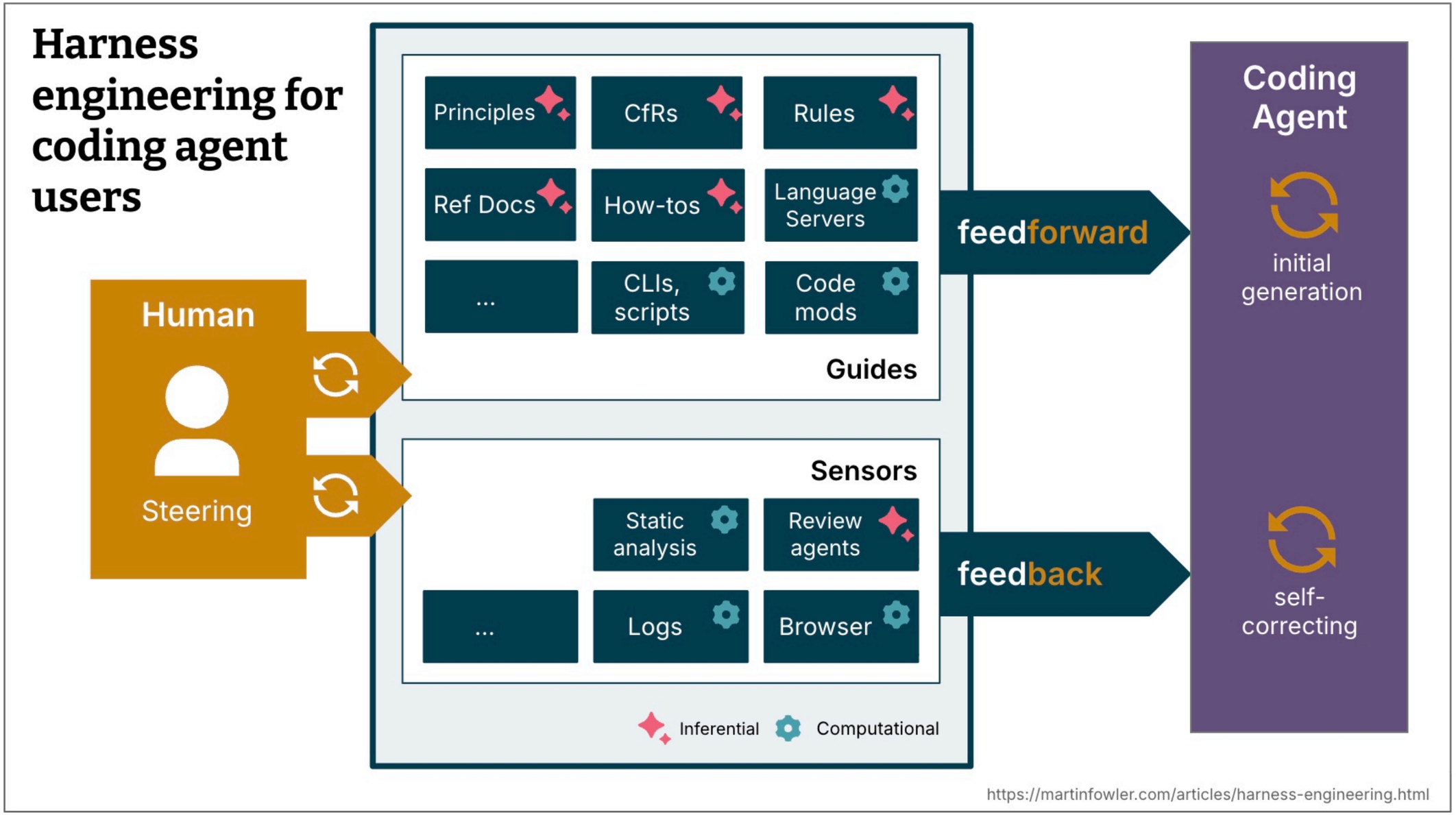
**Context**: Give a map, not a manual: OpenAI's experience is that AGENTS.md should be a **directory page**, not an encyclopedia.
- Stable Rules, Task-specific Skills, Retrieved Knowledge, Current Task State, Recent Execution Evidence

Constrain, don't micromanage: A good harness uses **executable rules** to constrain the agent, rather than enumerating instructions one by one.

**Keep Minimal** and Remove one at a time and observe: To quantify each harness component's marginal contribution, remove them one at a time and see which removal causes the biggest performance drop.

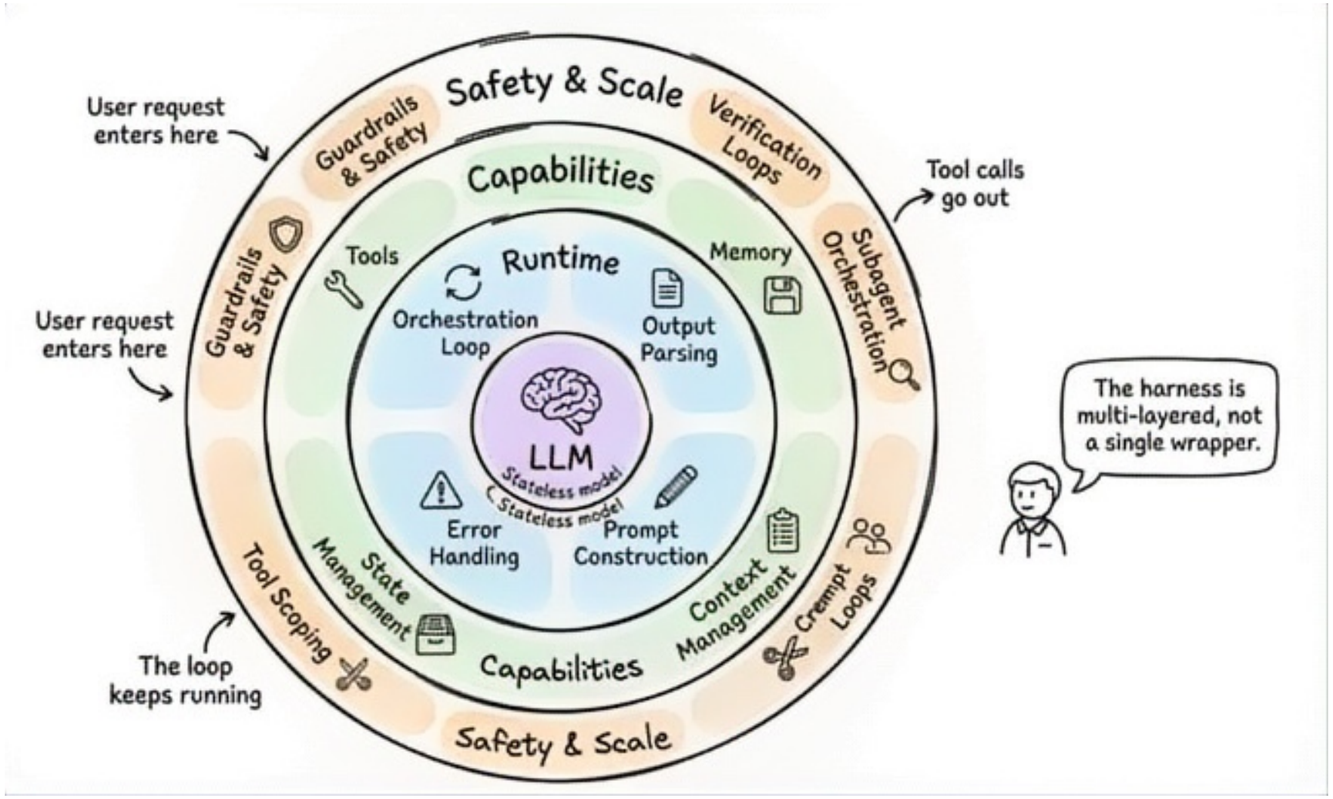
# Harness Concept: A Controlled Execution Framework for Agents




Harness engineering for coding agent users
Human
Steering
Principles
CfRs
Rules
Ref Docs
How-tos
Language Servers
...
CLIs, scripts
Code mods
Guides
Sensors
Static analysis
Review agents
...
Logs
Browser
Inferential
Computational
feedforward
feedback
Coding Agent
initial generation
self-correcting
https://martinfowler.com/articles/harness-engineering.html

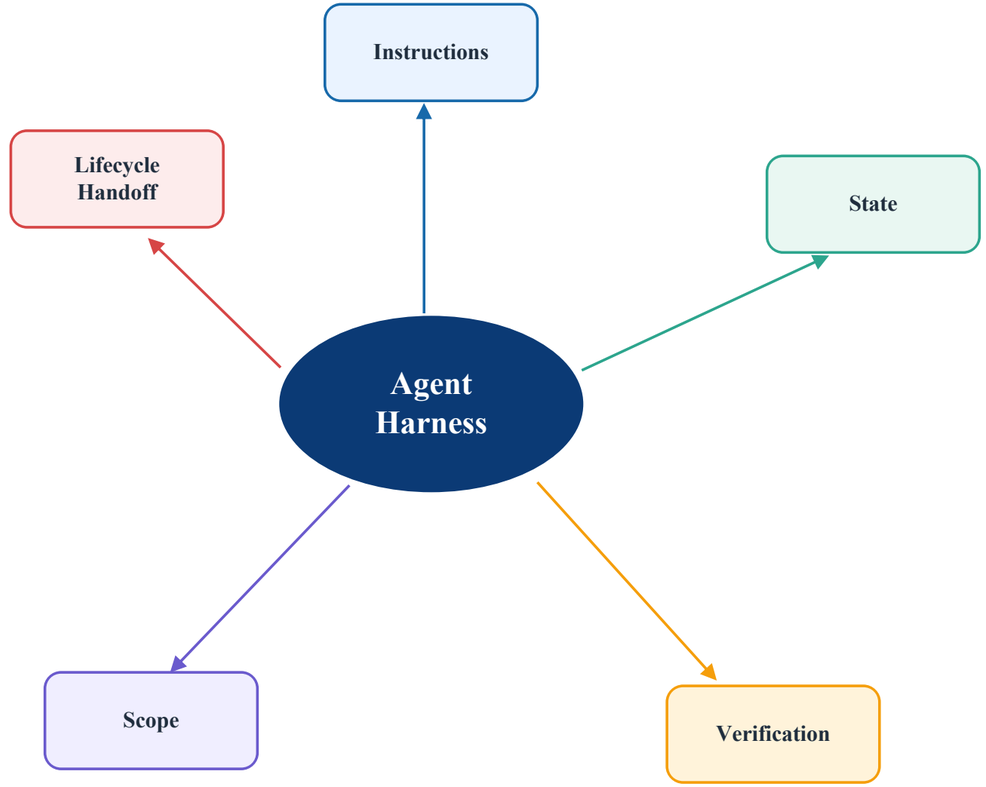
# Harness What Harness Needs to Control

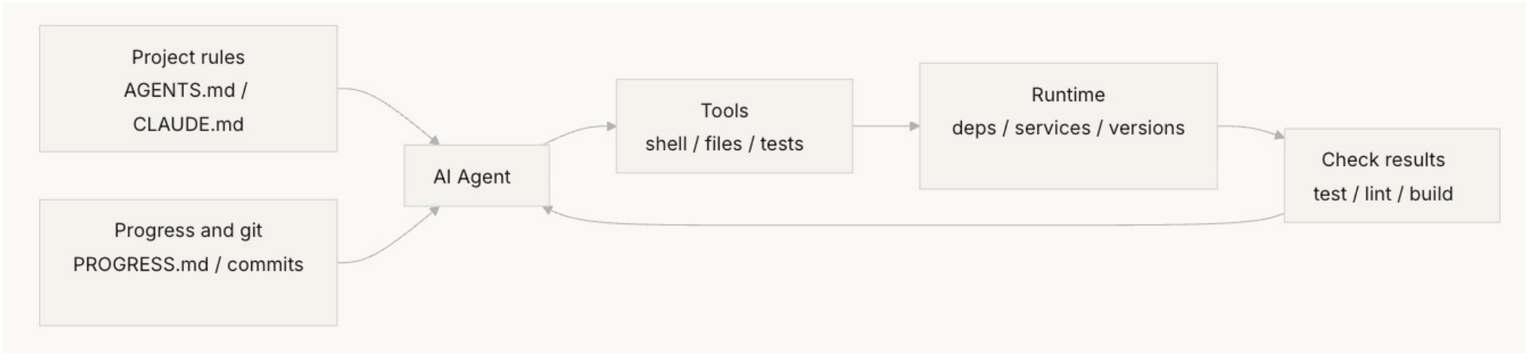
- **Instructions**
- context
- tools
- **Permission / Scope**
- **Verification**
- Approval
- **State**
- Lifecycle handoffs
- **Failure**
- Audit / Trace



# Five Core Harness Subsystems

- Instructions define rule priorities (prompts, skills)
- State records multi-step task status (Files)
- Verification ensures the process and results are verifiable (Logs, Browser, CI/CD)
- Scope limits task and resource boundaries (user permission, infra)
- Lifecycle Handoff decides when to transfer to a human or another Agent (Infra)



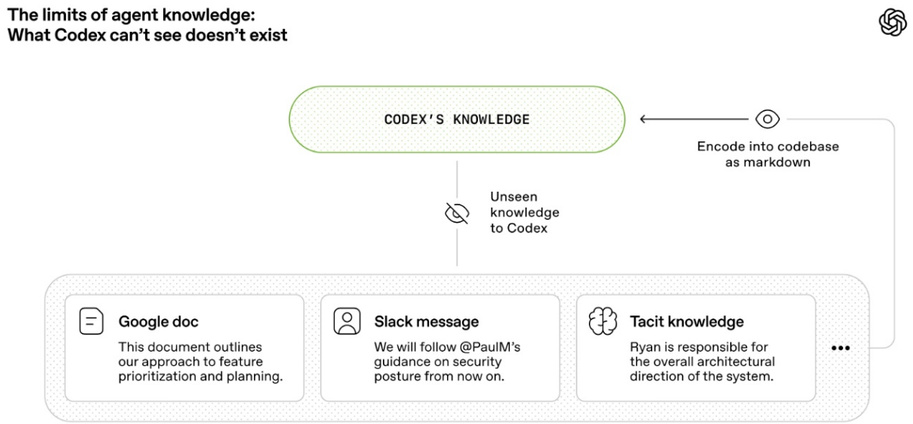



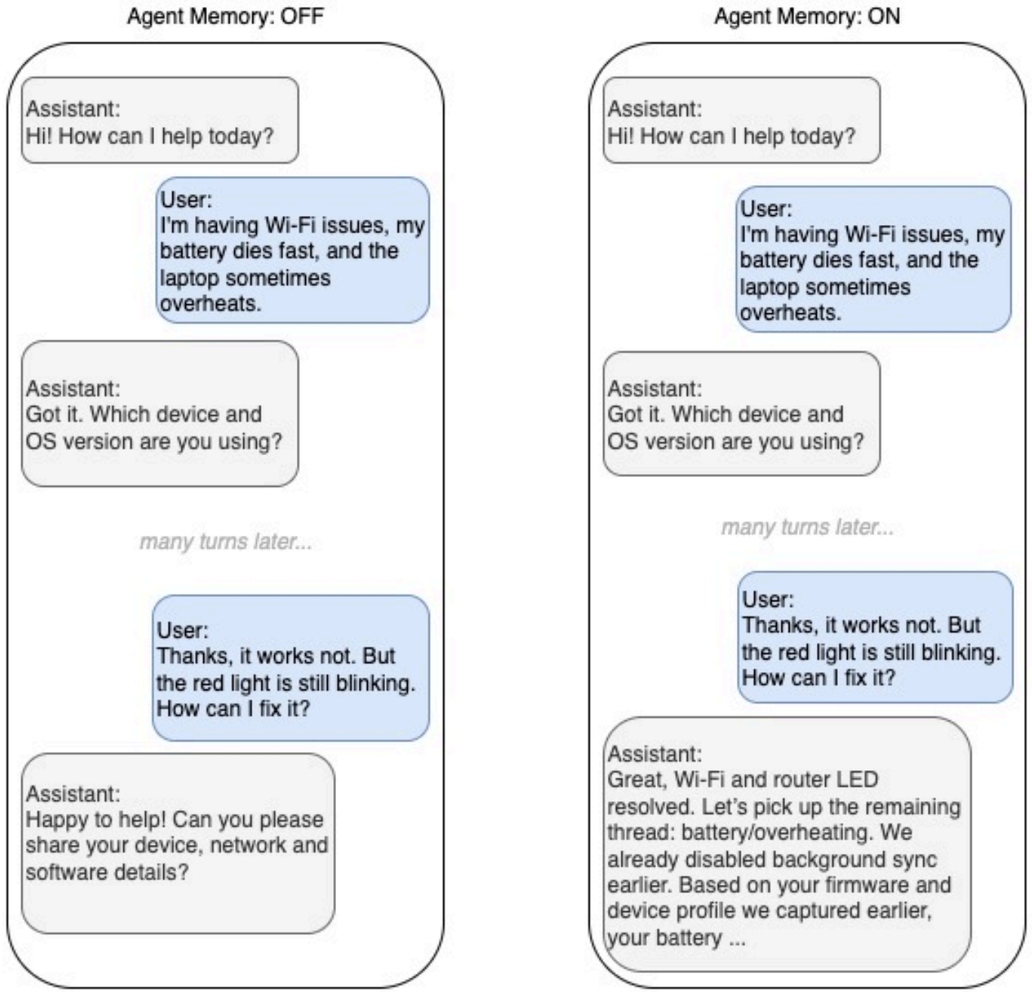
# Context Harness: controlling the Agent's context entry points

Context windows are finite

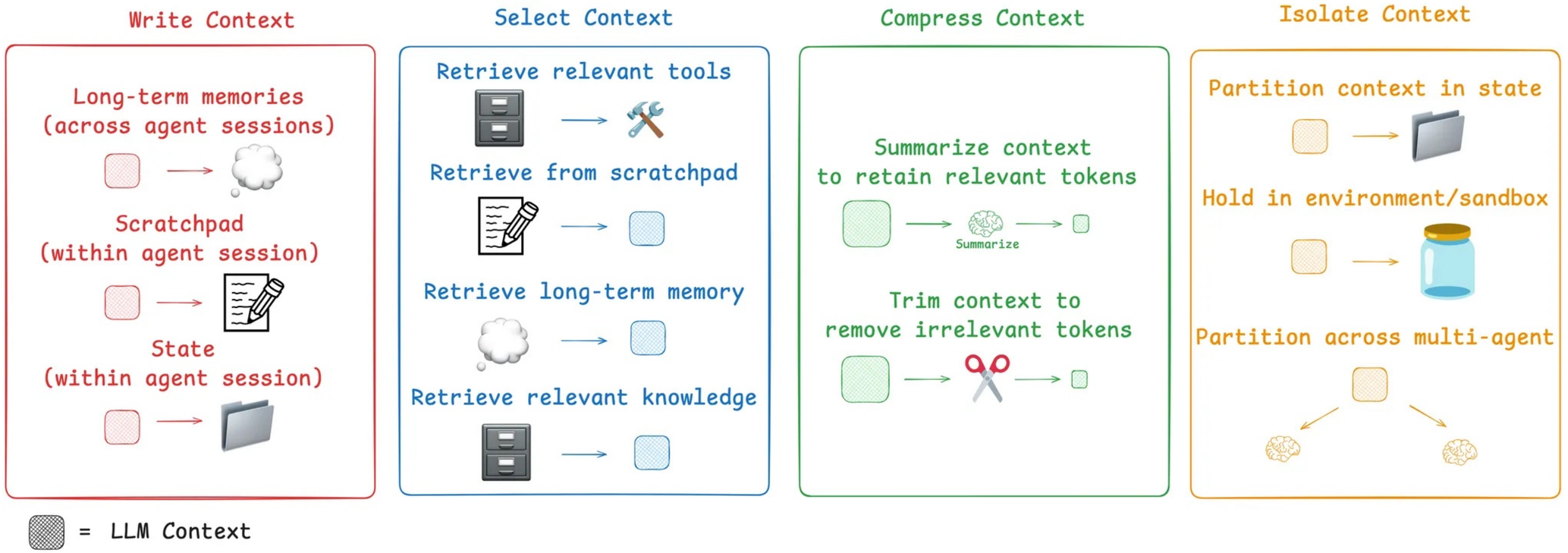
State persistence files: Persisted state files that let a new session unambiguously resume where the last one left off.

Context anxiety:  agents exhibit rushed finish behavior when approaching context limits, ending tasks early to avoid information loss. At its core, it's an irrational resource anxiety.



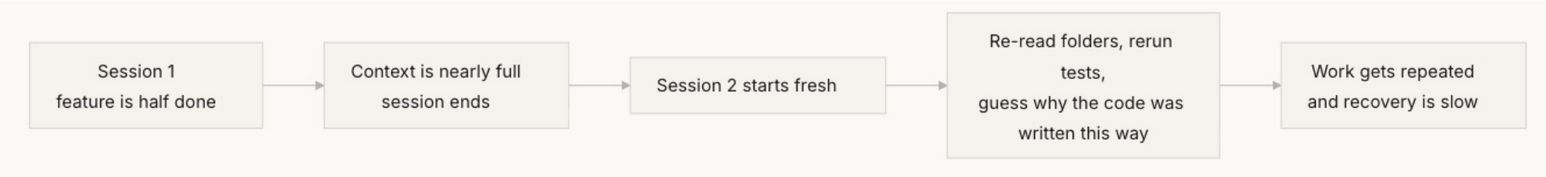



# Context Harness: controlling the Agent's context entry points

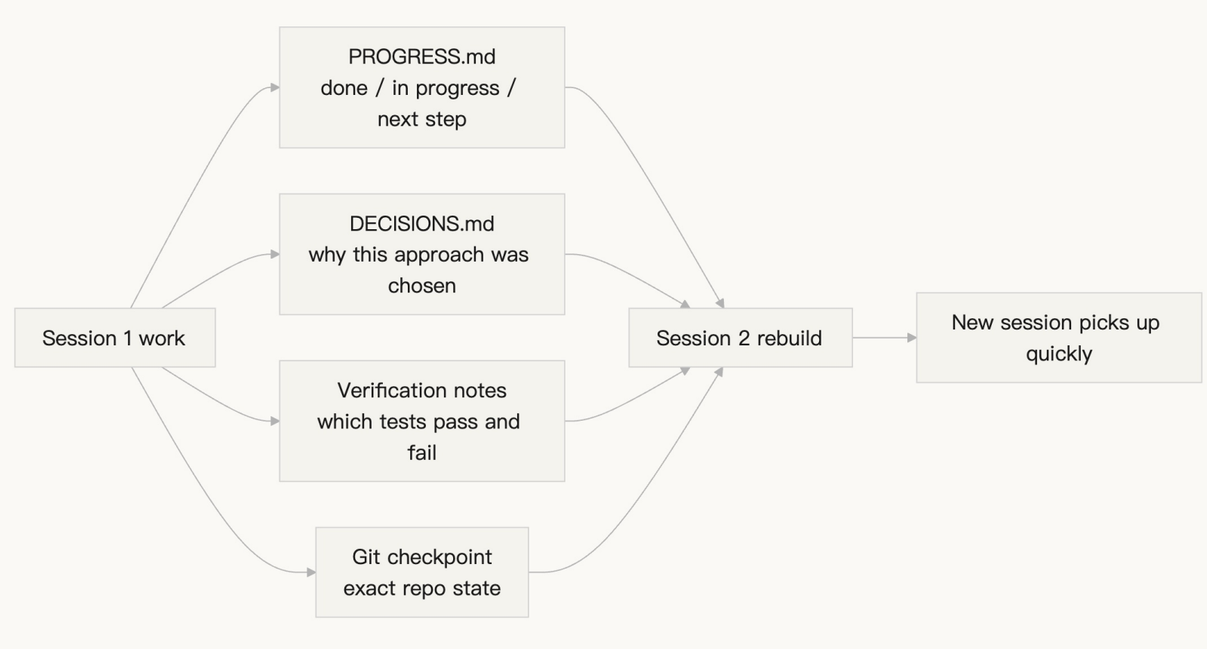


# Context Harness: controlling the Agent’s context entry points

Without state persistence files, every new session has to start from scratch:




With state persistence files, new sessions can pick up quickly:

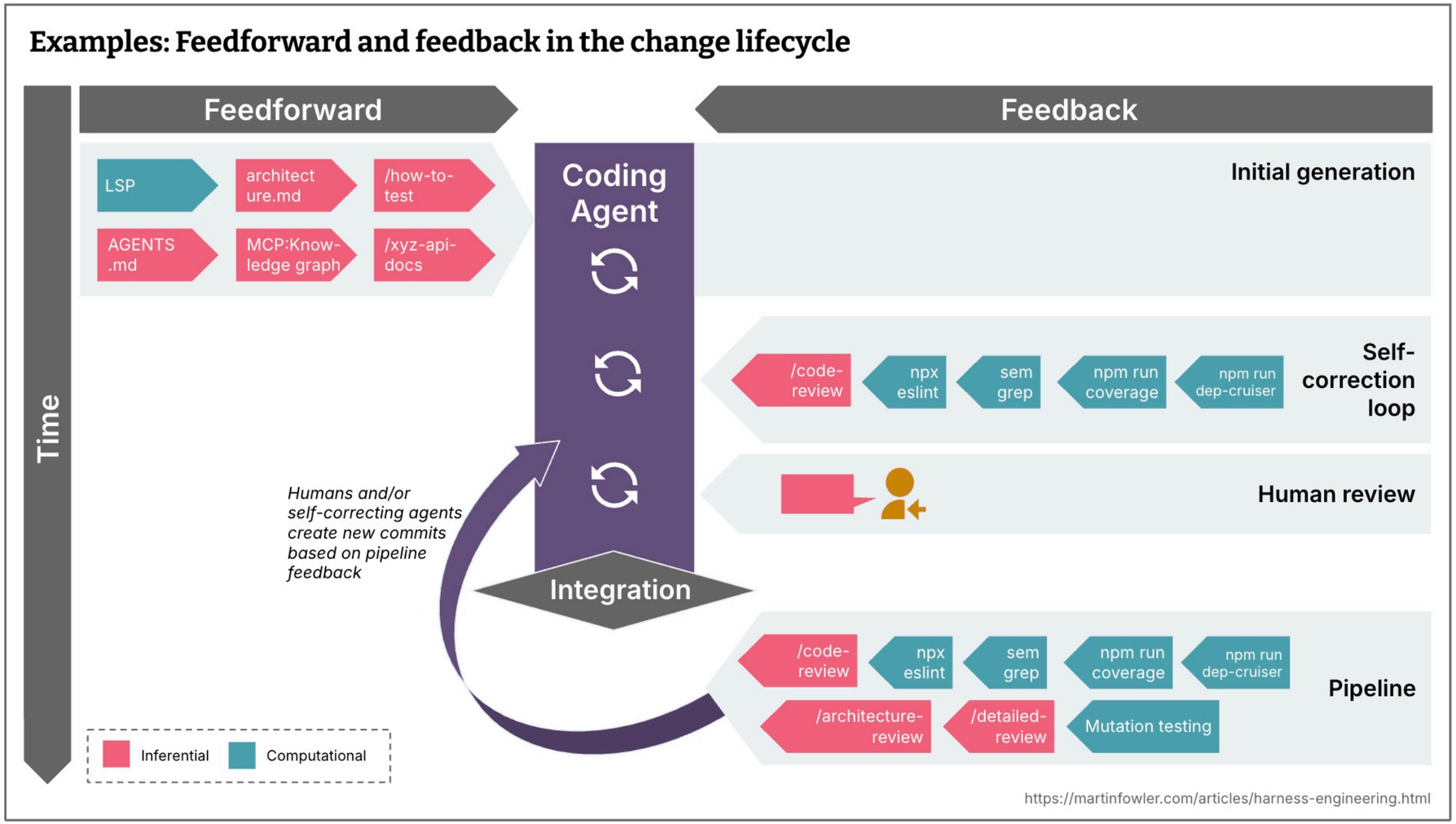


# Tool Harness

- Input Output Schema
- Auth
- Scope
- Timeout, Failure, Retry
- Idempotence
- Frequency Limit
- Error Code
- Auditable Fields
- Environment Isolation: dev / pre / prod

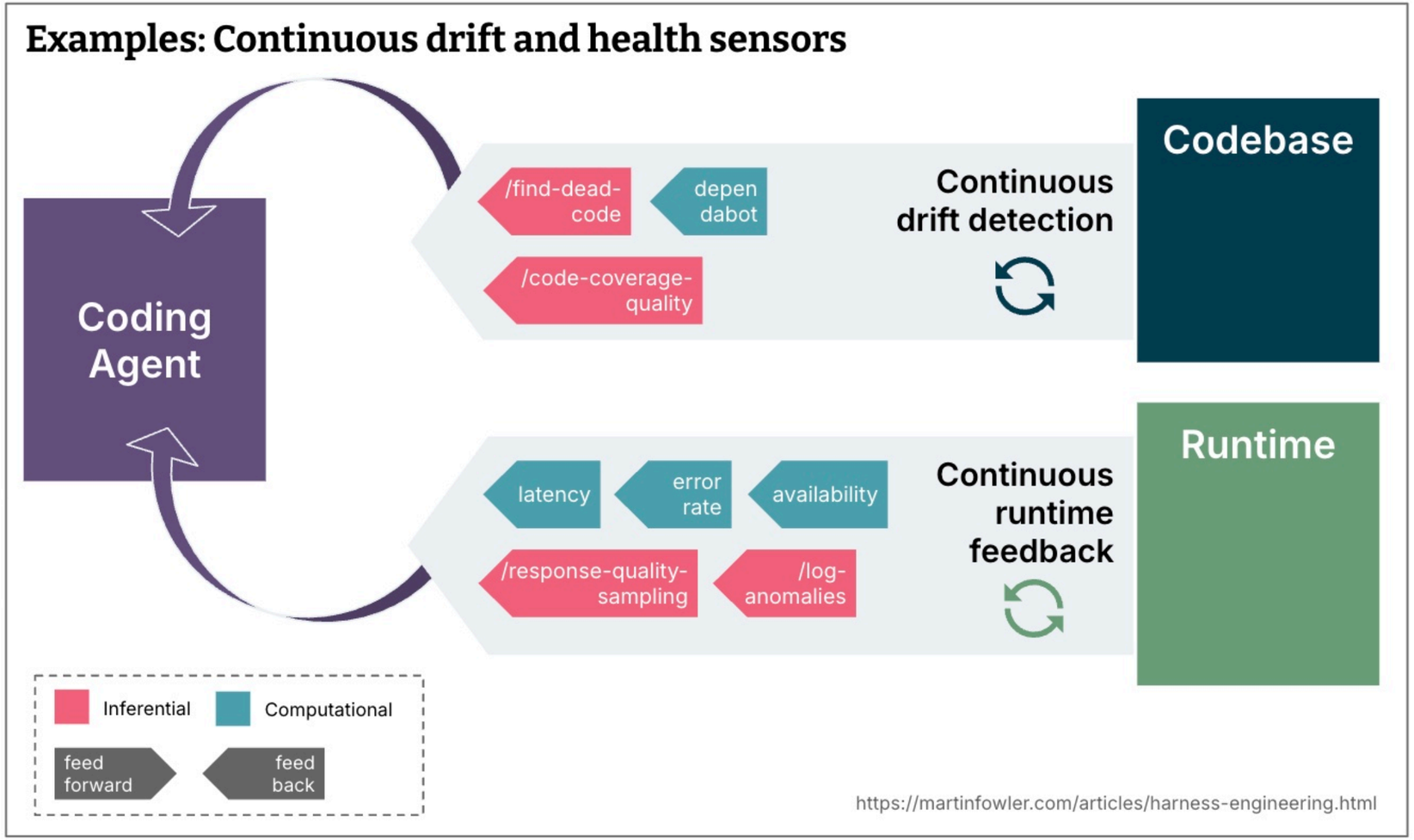
# Timing: Keep quality left

- What is reasonably fast and should be run even before integration, or even before a commit is even created? (e.g. linters, fast test suites, basic code review agent)
- What is more expensive and should therefore only be run post-integration in the pipeline, in addition to a repetition of the fast controls? (e.g. mutation testing, a more broad code review that can take into account the bigger picture)
- What if the environment doesn't support current developing?



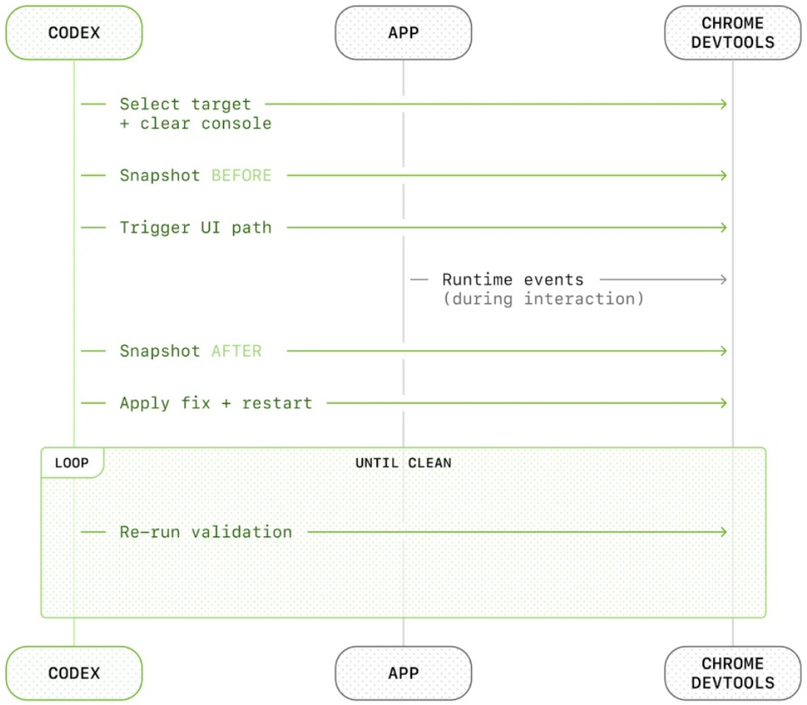
# Timing: Keep quality left

- 代码漂移（Code Drift）属于静态代码侧缓慢退化，不能只在代码提交时检查；需要常驻传感器持续巡检代码库，主动发现慢慢积累的技术债务
- 运行时反馈（Runtime Feedback）属于线上动态状态退化，让 Coding Agent 接入可观测数据，持续感知线上服务健康度下降、业务异常，从而主动生成优化修复方案。



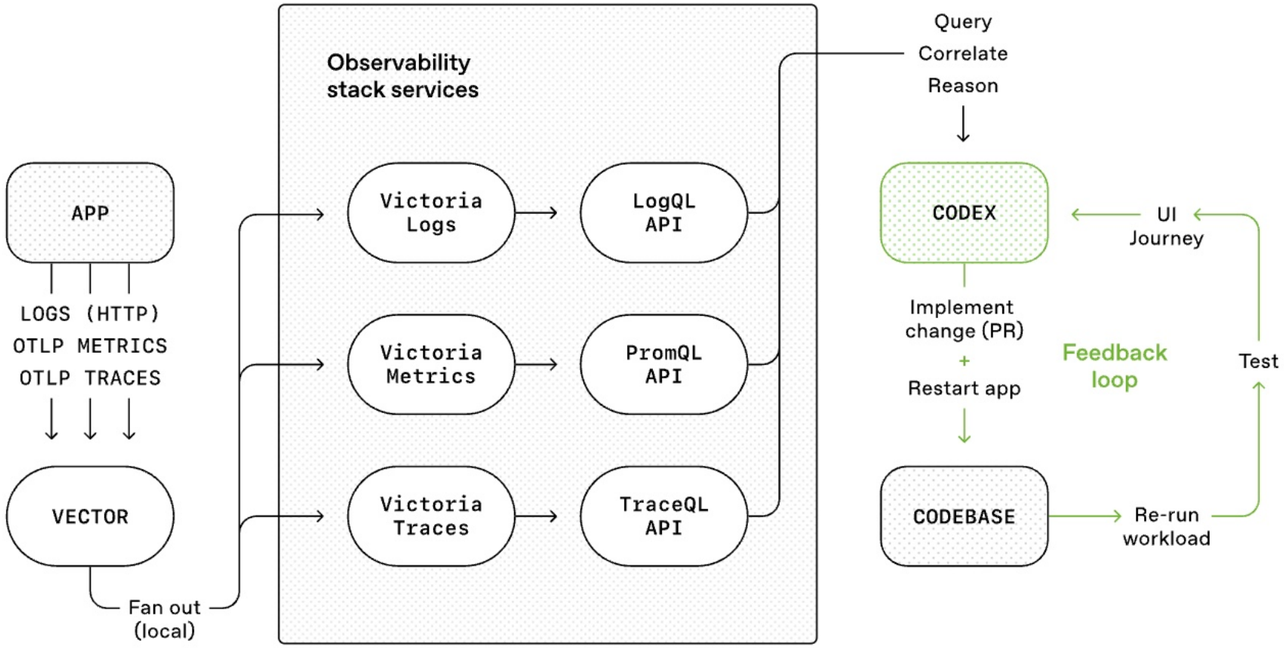
## Verification Harness

**Codex drives the app with Chrome DevTools MCP to validate its work**



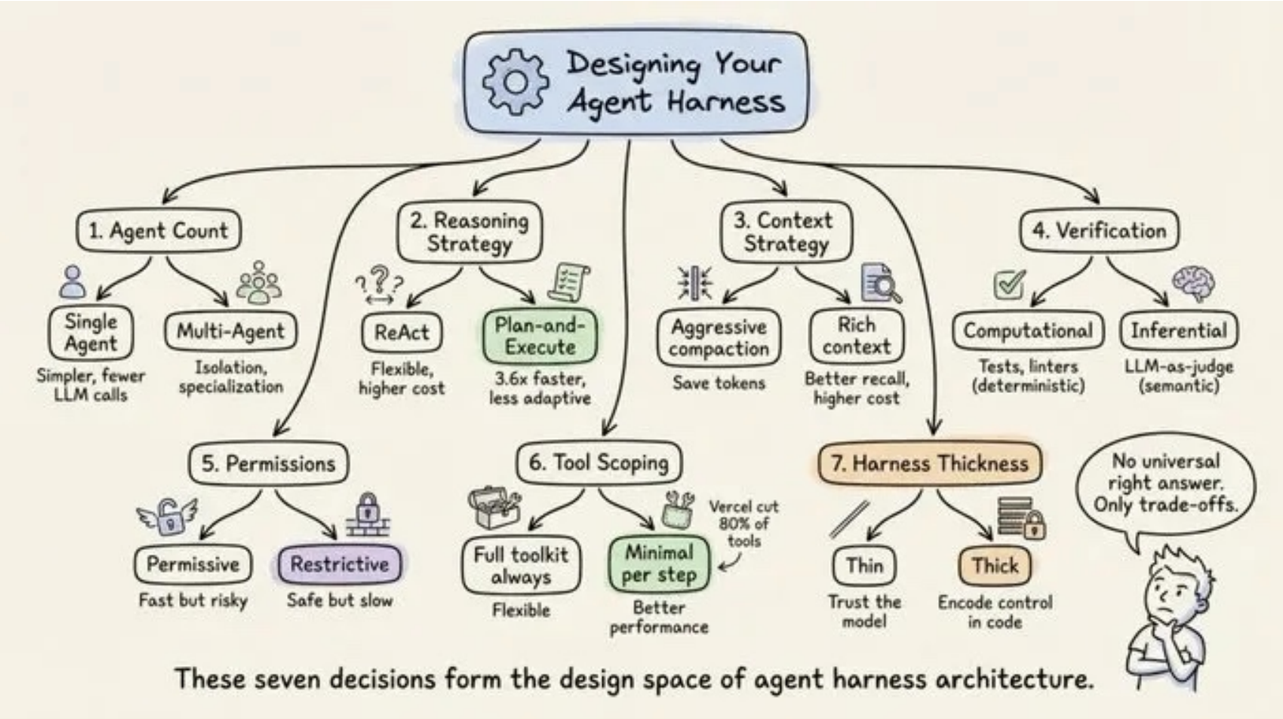

## Audit Harness

**Giving Codex a full observability stack**



## Seven decisions for Harness definitions

- **Single-agent** vs. multi-agent: single looping
- ReAct vs. **plan-and-execute**: todo-list
- Context window management strategy
- Verification loop design: **CI/CD** and **sem grep**
- Permission and safety architecture: user permission and Infra
- Tool scoping strategy: **shell, file, browser**
- Harness thickness: **Thin** -> Thick



# 4. Looping and Graph Engineering

# 5. Capability Ecosystem and Governance

# Skill Registry: Enterprise Capability Catalog

• The Skill Registry is the first catalog layer of the enterprise capability marketplace, registering each Skill's business scenario, owner, risk level, data needs, tool dependencies, knowledge version, evaluation results, and release status.

• Without a Registry, Skills scatter across personal documents, prompt repositories, low-code workflows, and private departmental projects, making them hard to reuse, evaluate, and retire.

• The Registry should support request, review, canary release, production release, version changes, deprecation, and rollback; every version change should have evaluation results and approval records.

• For Enterprises, a Skill owner is not a formal field; it is a maintenance responsibility when errors, policy changes, evaluation failures, or audit issues occur.

# Skill Registry: Enterprise Capability Catalog

# Tool Registry: Enterprise Tool Capability Catalog

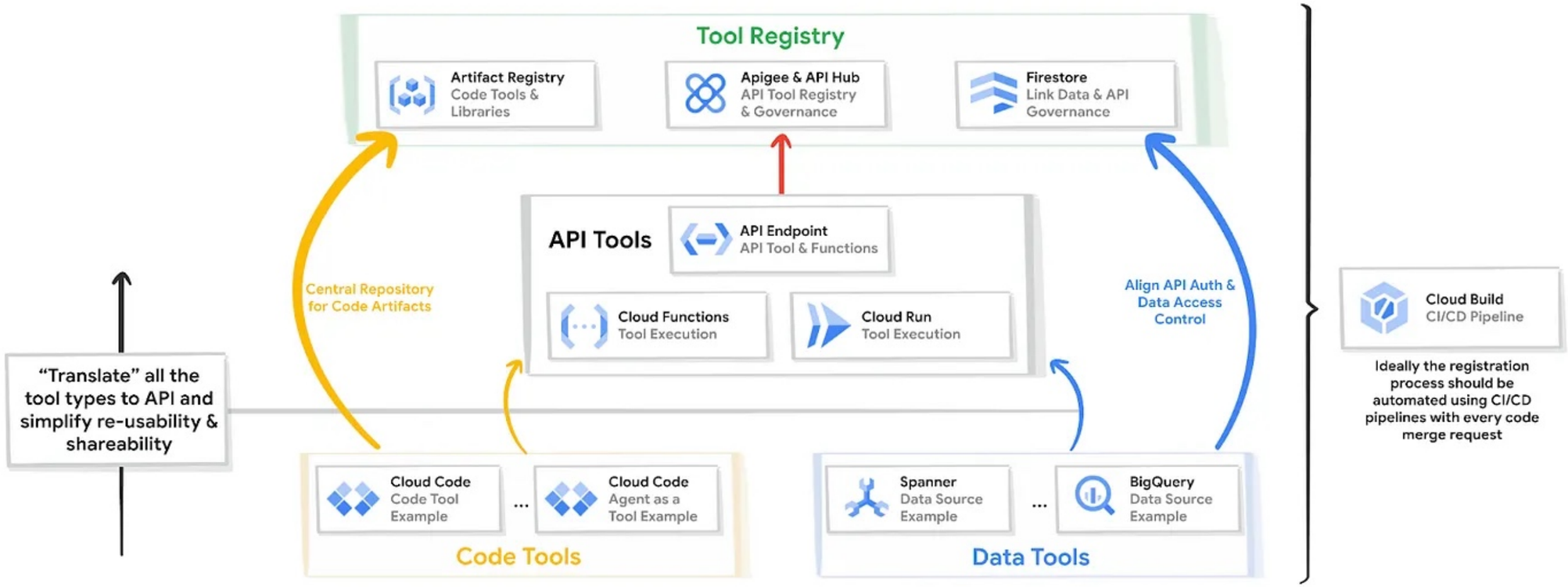
• The Tool Registry records each tool’s purpose, owning system, interface schema, permission model, data level, read/write type, invocation limits, rollback capability, and audit fields.

• Tools can connect systems, but they can also amplify risk. Without a unified catalog and authorization process, Agents can easily call outdated interfaces, private scripts, or unaudited internal APIs.

• The Tool Registry should distinguish read-only queries, sensitive reads, write operations, approval actions, and external notifications, and configure different Harness templates for each type.

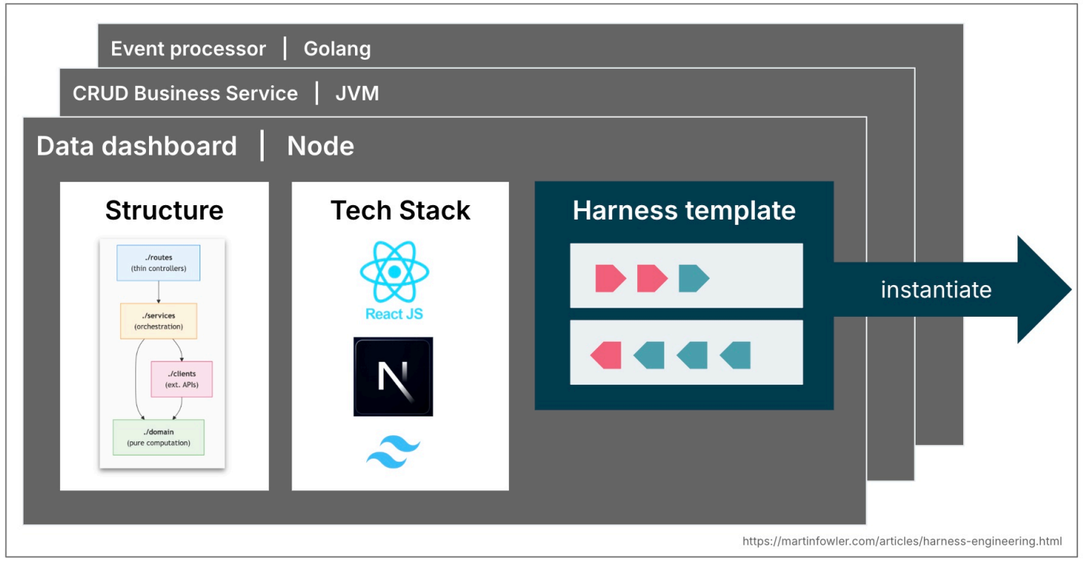
• Before release, tools must be tested for schema validation, permission checks, exception returns, idempotency, rate limiting, rollback, and completeness of audit fields.

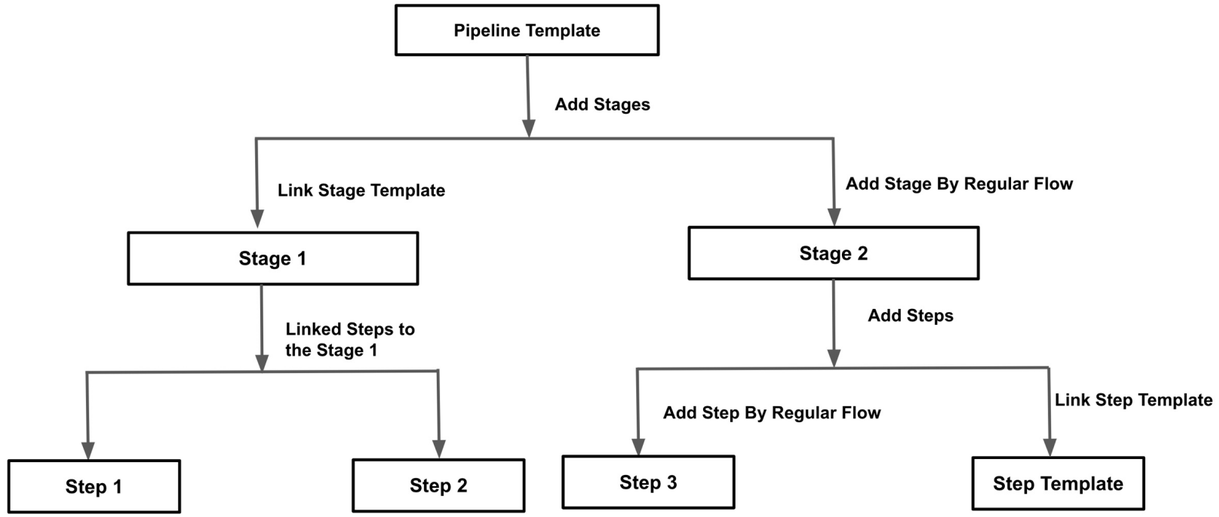
# Tool Registry: Enterprise Tool Capability Catalog



# Harness Templates: Turning Safety Controls into Templates

- A Harness Template packages context, tool, permission, approval, state, verification, failure, and audit controls into reusable templates, reducing the cost of redesigning safety for each Agent.
- Templates should be divided by risk level: read-only Q&A, read-only query, sensitive read, pre-approval before write operations, mandatory approval for high-risk tasks, sandbox execution, etc.



# Ecosystem Governance Item 1: Tiered Management

| Level | Type | Typical Scenarios | Data Access | System Write | Human Approval | Impact / Risk |
|---|---|---|---|---|---|---|
| L0 | Knowledge Q&A | Policy Q&A; training FAQ | Public / Low | No | No | Low |
| L1 | Internal Assistant | Meeting summary; document drafting | Internal Low | No | No | Low |
| L2 | Business Assistant | Customer info summary; analysis | Controlled business data | Read / draft only | Partial | Medium |
| L3 | Process Collaborator | Change review; case handling | Controlled business data | Limited | Yes | High |
| L4 | High-risk Decision Assistant | Credit approval assist; risk rating | Sensitive / critical data | No auto-write | Mandatory | Very High |

# Ecosystem Governance Item 2: Read/Write Permissions and Least Privilege

## Role Tiers

1. **Platform Admin**
   Global policies, platform configuration
2. **Domain Owner**
   Domain data, approved agent scope
3. **Agent Builder**
   Prompts, tools, workflows within scope
4. **End User**
   Use approved agents only

| | Read | Write | Approval Gate | Typical Control |
|---|---|---|---|---|
| **Platform Admin** | All governance metadata | Policies, permissions | Dual control | Global audit |
| **Domain Owner** | Domain knowledge, approved data | Domain configs, release approval | Owner approval | Scope boundary |
| **Agent Builder** | Dev docs, test data | Draft skills and evaluation cases | Review before release | Sandbox only |
| **End User** | Outputs and allowed context | No direct write | Human approval for risky actions | Least privilege |

## Core Governance Principles

- Least privilege by default
- Separate read and write rights
- Time-limited elevated access
- Full audit trail

# Ecosystem Governance Item 3: Data Classification

• Data classification determines whether an Agent can see data, how it queries it, whether it must be desensitized, whether it can output it, and whether audit is required. Without data classification, Context Harness and Permission Harness lack an execution basis.

| Level | Typical Data | Agent Rule | Control |
|---|---|---|---|
| 1 Public | Product brochures, public regulations | Reusable by approved agents | Basic logging |
| 2 Internal | SOPs, internal manuals | Use only in internal environments | Watermark + access log |
| 3 Confidential | Customer data, financial records | Use the minimum necessary data | Masking + role approval |
| 4 Restricted | Credentials, keys, payment instructions | No direct model exposure | Isolated tools + explicit approval |

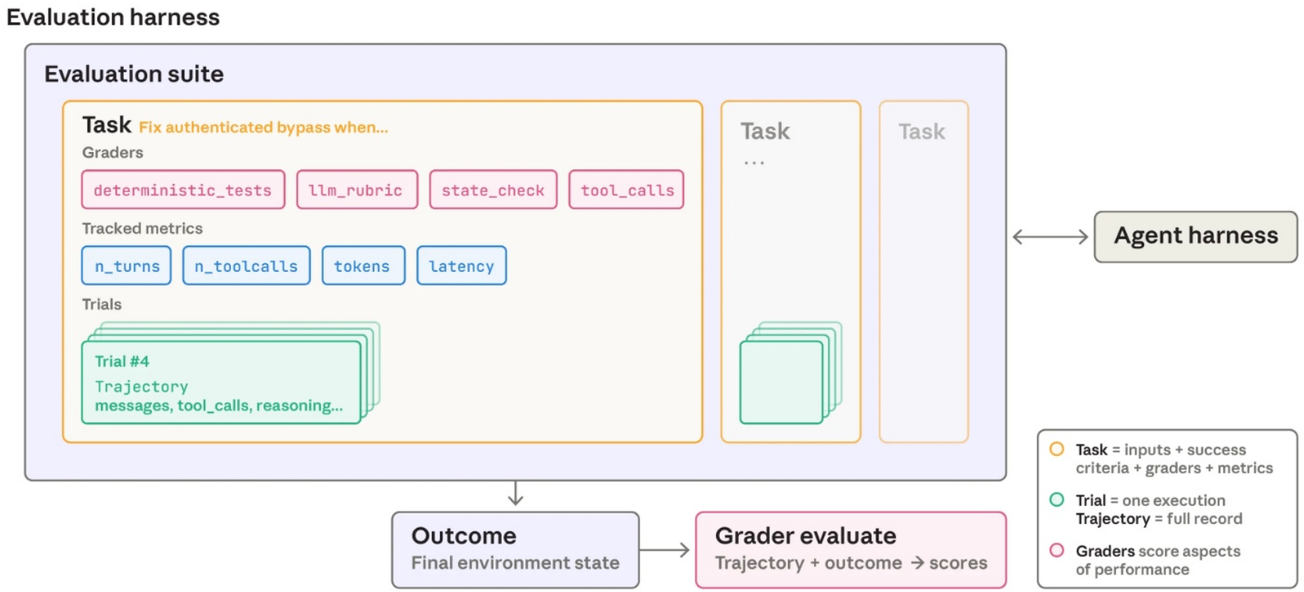
# Ecosystem Governance Item 4: Version Management

- Version management ensures that incident reviews are reproducible, changes are testable, and abnormalities are rollbackable.
- Objects that must be versioned include models, system instructions, Skills, knowledge bases, tool schemas, Harness Templates, evaluation sets, threshold policies, and approval rules.

# Ecosystem Governance Item 5: Evaluation System

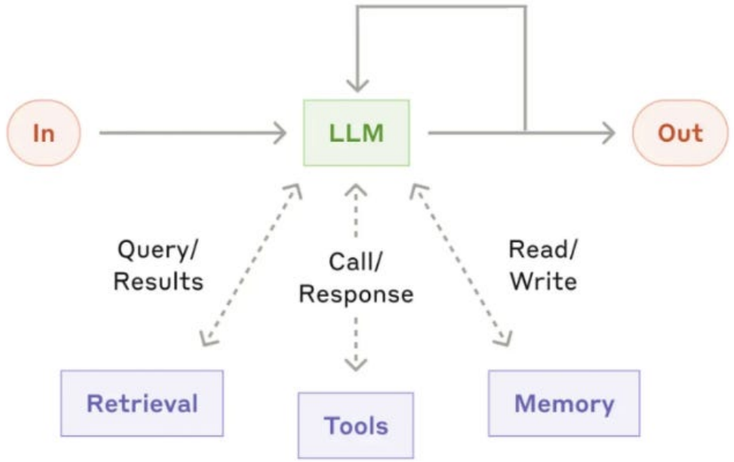
- Enterprise Agent evaluations should not focus solely on model capabilities but must encompass models, skills, tools, integration pipelines, and end-to-end workflows.

- Evaluation metrics should align with task objectives:
  - format compliance rate
  - citation accuracy rate
  - authorization override rejection rate
  - tool invocation accuracy rate
  - manual approval trigger rate
  - audit completeness rate
  - business adoption rate.

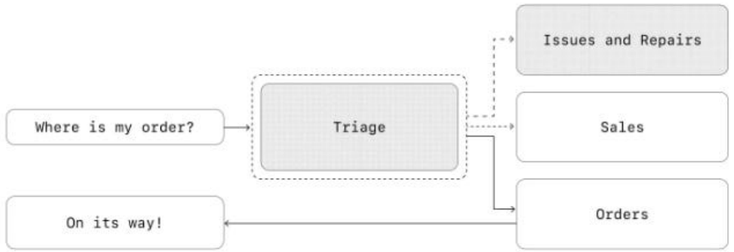


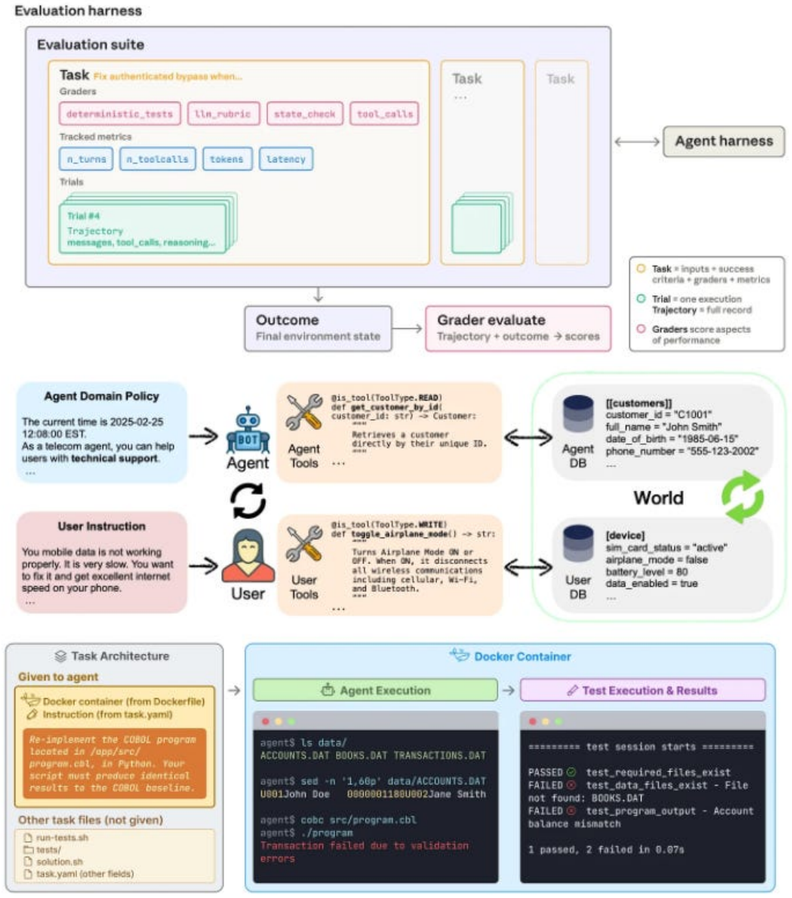
DEEP (LEARNING) FOCUS

# Agent Evaluation: A Detailed Guide





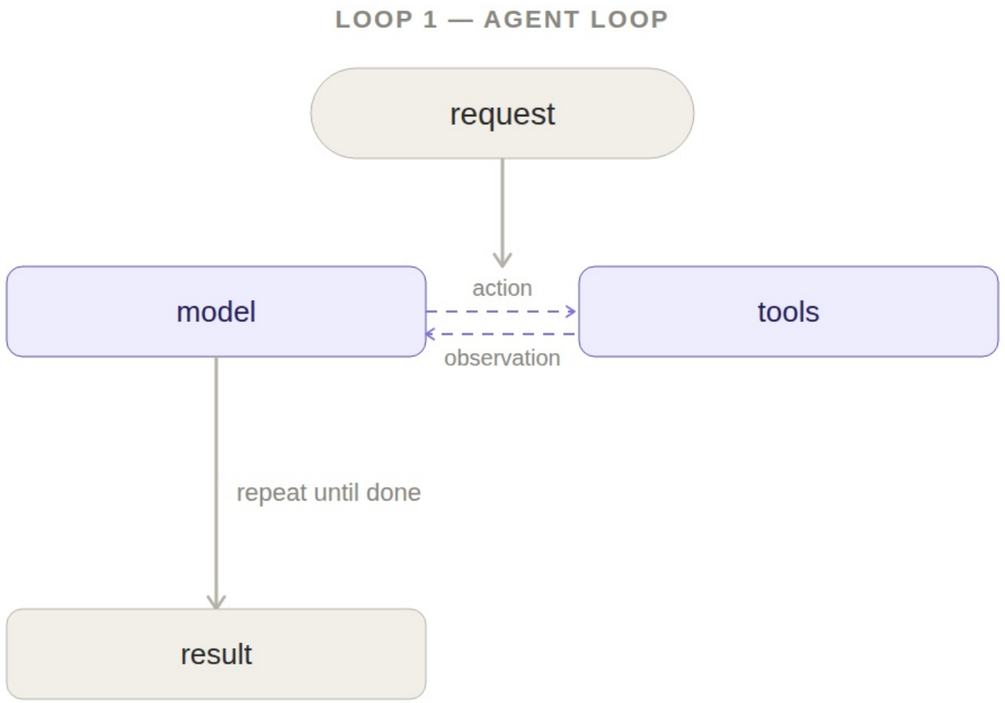



# Hands-on Practice

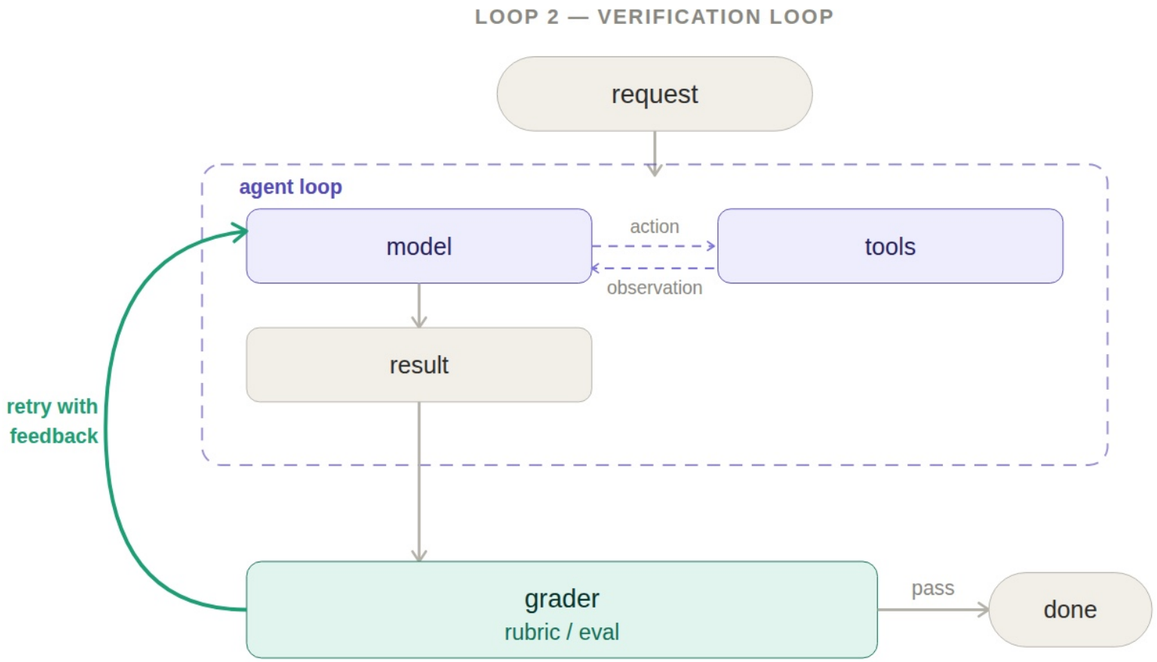
# Agent Loop

At its core, an agent is just a model calling tools in a loop until a task is complete.



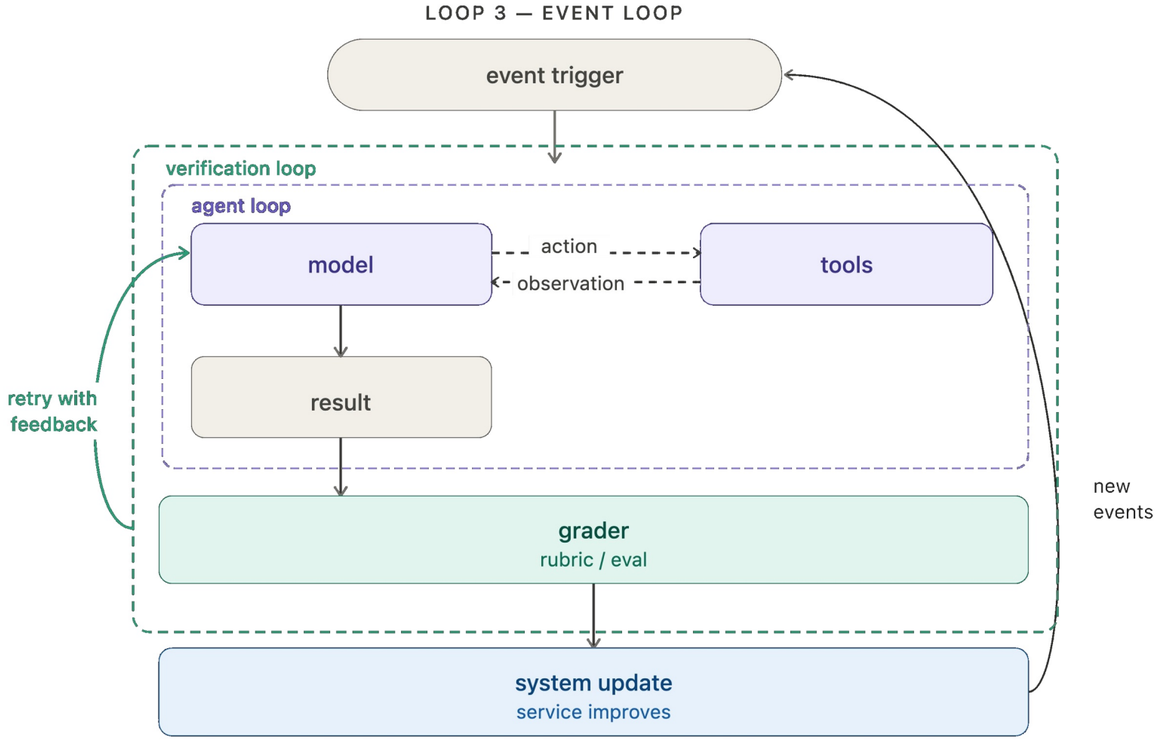
# Verification loop

The agent loop gets work done, but it doesn't always produce correct or consistent work on the first pass. When consistency matters, it's often useful to wrap it in a verification loop that checks the output and sends feedback back to the model when it falls short.



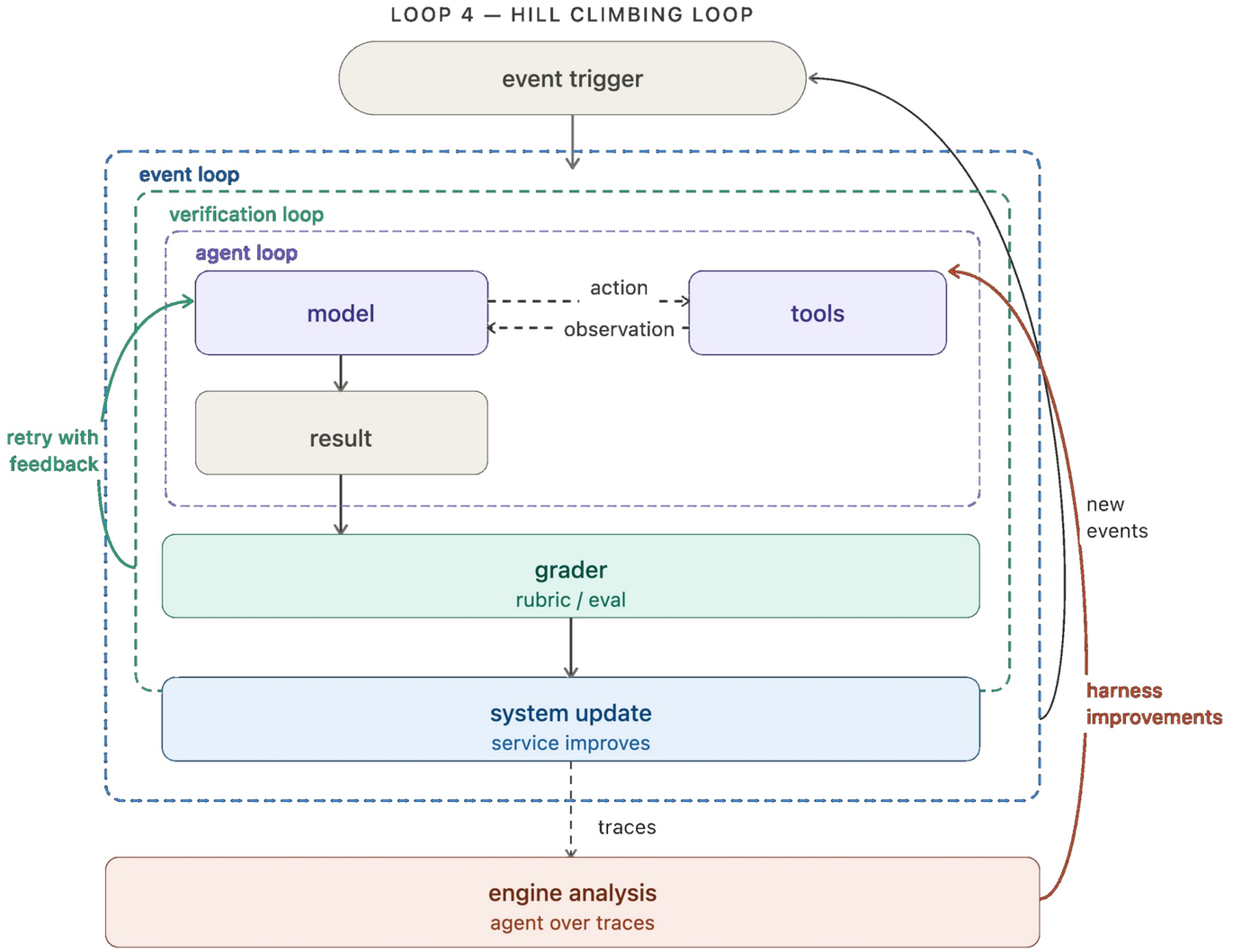
# Event driven loop

One of the most important parts of agent development is the integrations layer: connecting your agent to your ecosystem so that it can run in the background. The event-driven loop connects your agent to your ecosystem. An event fires — a new document lands, a schedule triggers, a webhook arrives — and the agent runs. The agent isn't something you invoke manually; it's a component running continuously inside a larger system.



# Hill climbing loop

Every agent run produce a trace: a record of what the model did, the tools it called, grader feedback, etc. Those traces contain high value signal regarding what's working and what isn't. The hill climbing loop runs an analysis agent over those traces and uses the findings to rewrite the harness with improved configuration. That can include prompt/tool tweaks or grader tweaks.



# Components of Agent Loops

**What is loop engineering**

- what is complete condition
- what is the next action
- how to save intermediate state
- what when fails
- when to continue / stop
- cost / time / risk

- **Automation**: heartbeat
- **Worktrees:** avoid chaos
- **Skills:** reusable package
- **Plugins and connectors: tools**
- **Sub-agents:** maker vs checker
- **State**: track what's done

| Primitive | Job in the loop | Codex app | Claude Code |
|---|---|---|---|
| Automations | discovery + triage on a schedule | Automations tab: pick project, prompt, cadence, environment; results land in a Triage inbox; `/goal` for run-until-done | Scheduled tasks and cron, `/loop`, `/goal`, hooks, GitHub Actions |
| Worktrees | isolate parallel features | Built-in worktree per thread | `git worktree`, `--worktree`, `isolation: worktree` on a subagent |
| Skills | codify project knowledge | Agent Skills (`SKILL.md`), invoked with `$name` or implicitly | Agent Skills (`SKILL.md`) |
| Plugins / connectors | connect your tools | Connectors (MCP) plus plugins for distribution | MCP servers plus plugins |
| Sub-agents | ideate and verify | Subagents defined as TOML in `.codex/agents/` | Task subagents in `.claude/agents/`, agent teams |
| State | track what's done | Markdown or Linear via a connector | Markdown (`AGENTS.md`, progress files) or Linear via MCP |

# Bad Agent Loops

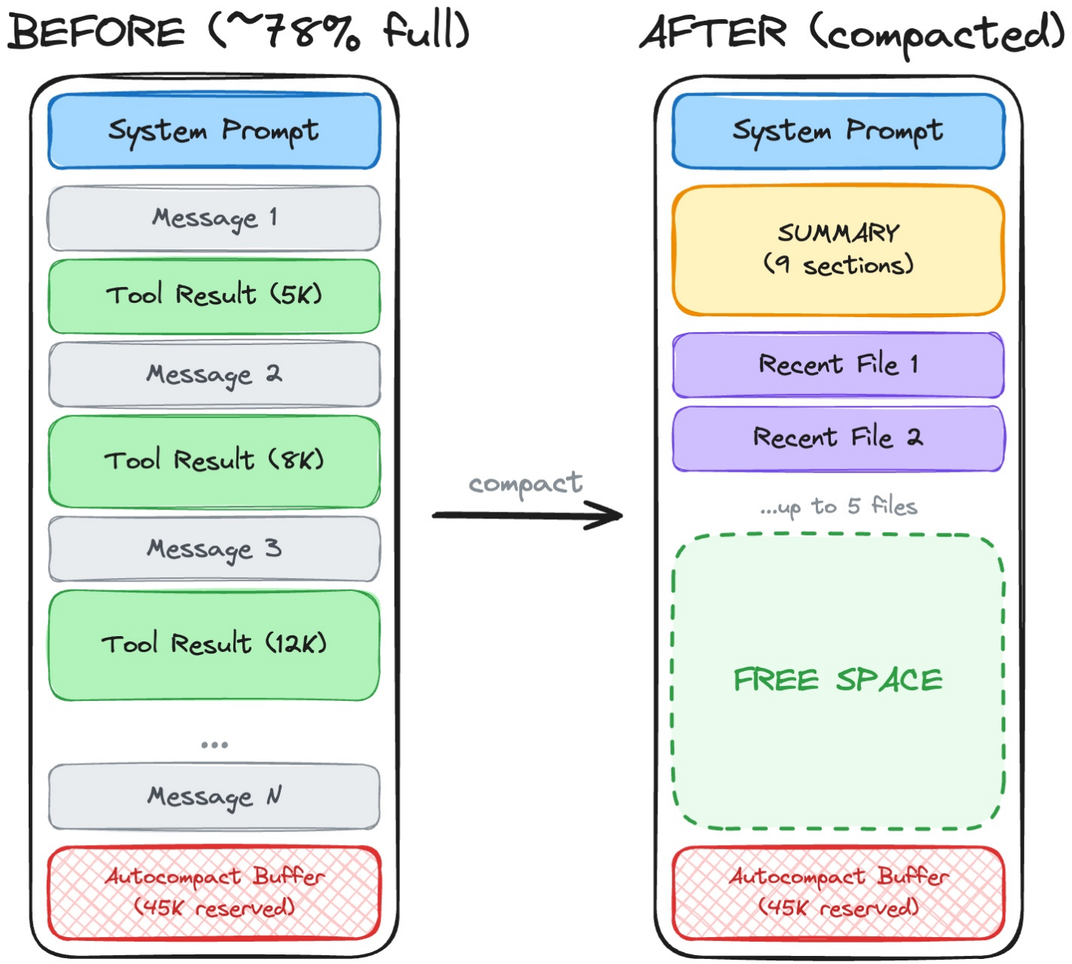
**Good Agent Loops always cost more tokens, but should be controllable.**

- Infinite looping: no max-iteration, cost control, time control
- Self-Verification: no independent sub-agents for maker and checker
- Mechanical retry: repeat the same operation after failure
- No Explicit State: the agent always guess the status of the task from the entire execution context
- No Explicit Success Criteria: unclear how to complete a task
- Verification produces no effects on Execution: Checker runs tests but Maker makes no changes
- No Fallback / Blocked Status: Continues to operate automatically even under high-risk or prolonged failure conditions.

# Context Compaction

| Approach | What it does |
| --- | --- |
| **Truncation** | Cut old messages. Simple but lossy. |
| **Summarization** | Condense conversation into summary. Preserves meaning but loses detail. |
| **Compaction** | Summarize + restore recent files + preserve todos + inject continuation instructions. |

# Context Compaction




- user intent (what was asked for, what changed)
- key technical decisions and concepts
- files touched and why they matter
- errors encountered and how they were fixed
- pending tasks and the exact current state
- a next step that matches the most recent user intent

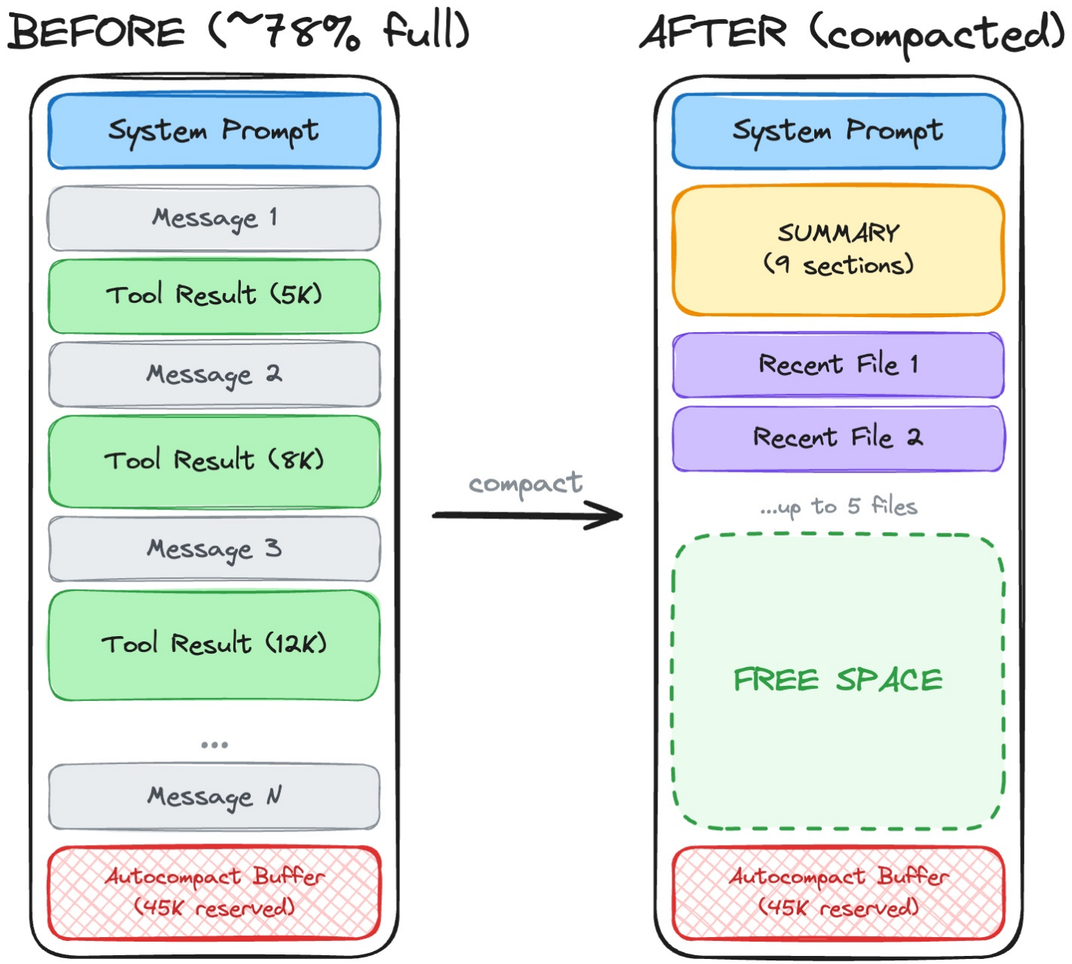
After summarizing, Claude Code rebuilds context with:

1. **Boundary marker** — marks the compaction point
2. **Summary message** — the compressed working state
3. **Recent files** — re-read the few most recently accessed files
4. **Todo list** — preserve task state
5. **Plan state** — if you were in a "plan required" workflow
6. **Hook outputs** — if startup hooks injected context

```
❯ /context
  └ Context Usage
    ⛁ ⛀ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁   claude-opus-4-5-20251101 · 101k/200k tokens (50%)
    ⛁ ⛁ ⛁ ⛀ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁
    ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁   Estimated usage by category
    ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁   ⛁ System prompt: 3.3k tokens (1.6%)
    ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁ ⛁   ⛁ System tools: 16.2k tokens (8.1%)
    ⛁ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ Memory files: 6.0k tokens (3.0%)
    ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶   ⛁ Skills: 1.4k tokens (0.7%)
    ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛶ ⛒ ⛒   ⛁ Messages: 74.4k tokens (37.2%)
    ⛒ ⛒ ⛒ ⛒ ⛒ ⛒ ⛒ ⛒ ⛒ ⛒   ⛶ Free space: 54k (26.9%)
    ⛒ ⛒ ⛒ ⛒ ⛒ ⛒ ⛒ ⛒ ⛒ ⛒   ⛒ Autocompact buffer: 45.0k tokens (22.5%)
```

You are given a few messages from a conversation, as well as a summary of the conversation so far. Your task is to summarize the new messages based on the summary so far. Aim for 1-2 sentences at most, focusing on the most important details.

# Context Compaction




Claude Code's compaction system:

1. **Microcompaction** — preserve a small hot tail, offload bulky outputs to disk
2. **Auto-compaction** — trigger compaction when free space falls below reserved headroom
3. **Structured summarization** — preserve intent, decisions, errors, and "what to do next"
4. **Rehydration** — re-read recent files and restore todo/plan state so work continues cleanly
5. **File restoration** — re-reads your 5 most recent files after summarizing
6. **Continuation message** — tells Claude to resume without re-asking what you want

# Context Compaction

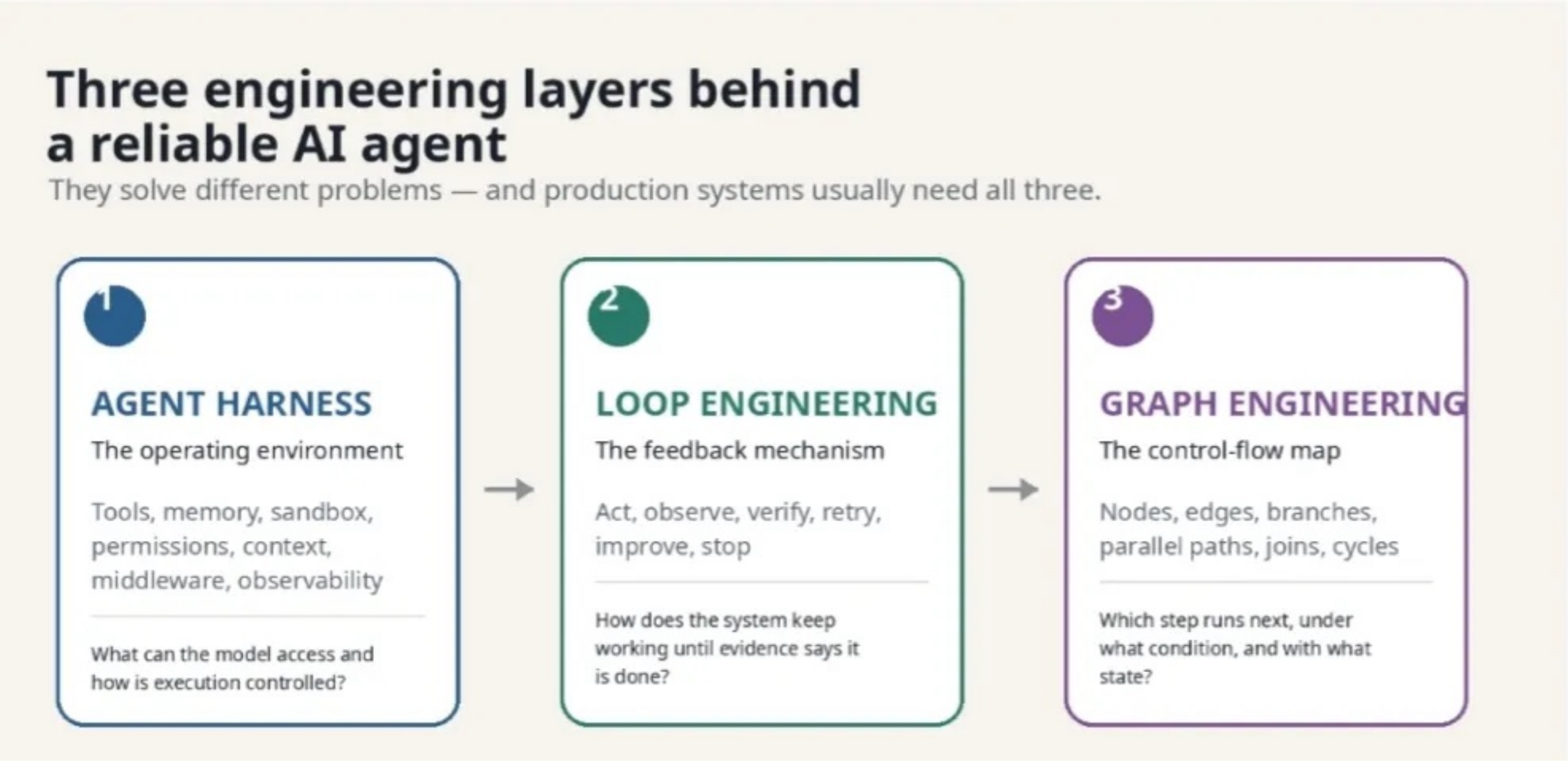
**Codex**
- Handoff summary: progress, constraints, remained tasks
- **Maintain the original user message**
- Summarize agent response and tool results

**OpenCode**
- Prune: hide old context but not delete the data
- Summary: 5-section summaries; start from the latest user request but not start from summary

**Cursor / Cline**
- Notify the user to create a new chat session
- Start from the summary of the old one

**Agent Teams**
- Sub-Agent start from the hand-off summary of the Parent-Agent

# Loop Engineering vs. Graph Engineering



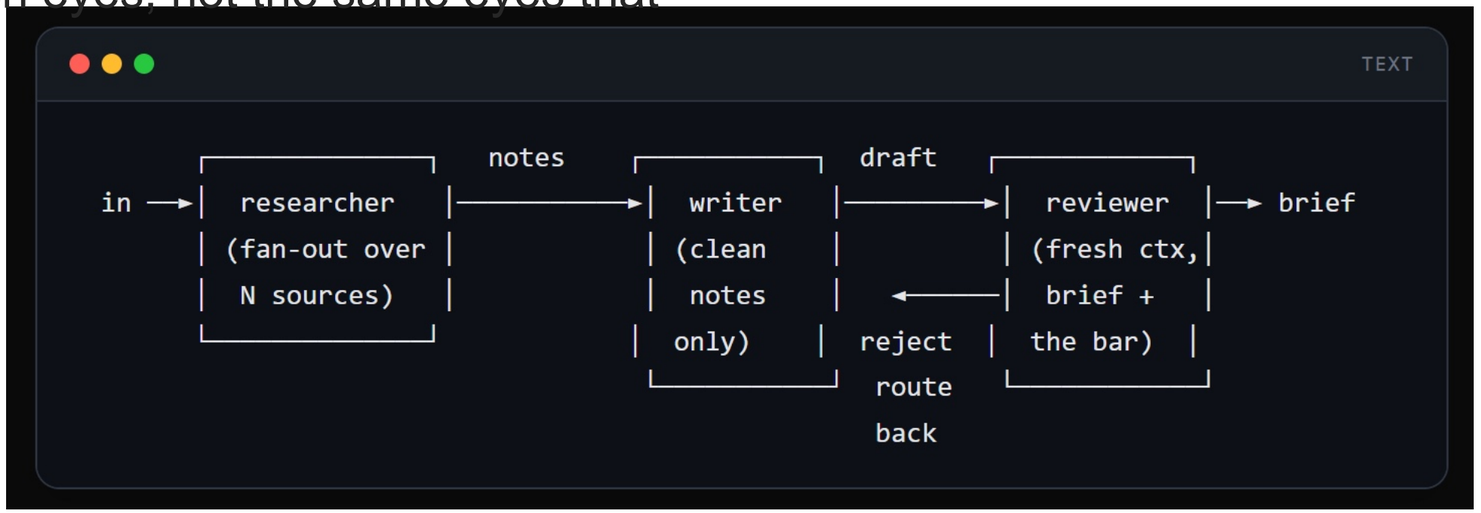
# Loop Engineering vs. Graph Engineering

| | Loop engineering | Graph engineering |
|---|---|---|
| **Unit** | One agent's cycle | Multiple nodes + edges + shared state |
| **Shape** | A cycle: discover, plan, execute, verify, repeat | A directed graph: nodes wired by routing edges |
| **Nodes** | N/A (it *is* the loop) | Specialized agents or steps (researcher, writer, reviewer) |
| **Edges** | The single "repeat" arrow back to the start | Control flow between nodes: branches, fan-out/fan-in, loops |
| **State** | Lives inside one agent's context | Flows *along the edges*, shared between nodes |
| **You design** | The cycle and its stop condition | The topology and the routing |
| **Fails when** | The verifier is weak | The topology is wrong, or state leaks between nodes |

# Loop Engineering vs. Graph Engineering

## Role-aware DAG

- The researcher node fans out across the sources in parallel and returns structured notes. It never writes prose.
- The writer node sees only clean notes, never the raw HTML, and produces the brief.
- The reviewer node runs in a fresh context, sees only the brief and the accuracy bar, and routes back to the writer if it fails. Fresh eyes, not the same eyes that wrote it.



# 4. Case

# Case Input: Employee Training Platform Meeting Discussion

RAW MEETING TEXT

"Today we discussed the employee  AI training system. Everyone agreed that the first version should be an MVP, supporting uploaded training materials and automatic generation of course outlines and quiz questions. Zhang San is responsible for backend APIs, Li Si for frontend pages, and Wang Wu for test cases. The team hopes to finish the first version by next Wednesday. Two issues remain: API Key how API Keys should be distributed is still unclear, and the on-site network may be unstable, so an offline plan is needed."

messy natural-language input

POTENTIAL OUTPUTS

### Meeting Topic

Employee AI training system development planning and functional design discussion

### Feature Ideas

Upload training materials, automatically generate course outlines, and intelligently generate quiz questions

### Task Assignment

Zhang San handles backend APIs; Li Si handles frontend pages; Wang Wu handles test cases

### Deadline

Deliver the first version by next Wednesday as the MVP milestone

Meeting Minutes | Requirements | Test Cases | Decision Memo

# Case Preparation:from “can generate content” to “can complete tasks in a controlled and auditable way”

| Skill | Role | Applicable Scenarios |
| --- | --- | --- |
| Meeting Notes Skill | organize meeting minutes | meetings, discussions, transcripts |
| Requirements Breakdown Skill | break down requirements | product, project, business requirements |
| Test Case Generation Skill | generate test cases | QA, development, acceptance |
| Decision Memo Skill | generate decision memos | option comparison, management reporting |

1

**Meeting Notes Skill**

organize meeting discussions into standard meeting minutes

↓

2

**Requirements Breakdown Skill**

break meeting conclusions into product requirements

↓

3

**Test Case Generation Skill**

convert requirements into test cases

↓

4

**Decision Memo Skill**

organize options and risks into a decision memo

↓

5

**Harness security validation layer**

Input Guardrail | Intent Classification | Skill Routing | Output Validation | Human Review Gate | Audit Logger

↓

6

**Harness Validation**

input guardrail → intent classification → skill routing → output validation → human review → audit logging

↓

7

**Final Output**

Raw Text → Meeting Notes → Requirements → Test Cases → Decision Memo → Harness Validation → Final Output

# Skill 1：Meeting Notes Skill

## Role

Turn messy meeting discussions into standard meeting minutes

## Applicable Scenarios

meeting records
voice transcripts
chat discussions
project sync notes

## Problem Solved

~~Without Skillwhen used:LLMmay only give a simple summary:"everyone discussed the employee training system and assigned tasks"~~

**With Skillwhen used:**the output always includes6structured fields

With Skillstandard output

**Meeting Topic**

**Key Discussion Points**

**Decisions Made**

**Action Items**

**Open Questions**

**Risks / Blockers**

# Meeting Notes Skill Output Effect

## Meeting Minutes

### 1 Meeting Topic

Employee AI Training Platform MVP

### 2 Key Discussion Points

- Build a minimum viable version first
- Support uploading training materials
- Automatically generate course outlines and quiz questions
- Prepare an offline option because the network may be unstable

### 3 Decisions Made

- Start with an MVP instead of a full platform
- Target initial completion by next Wednesday

### 4 Action Items

| Task | Owner | Deadline | Status |
| --- | --- | --- | --- |
| Backend API development | Zhang San | Next Wednesday | Planned |
| Frontend page development | Li Si | Next Wednesday | Planned |
| Test case preparation | Wang Wu | Next Wednesday | Planned |

### 5 Open Questions

- How should API Keys be distributed?
- How should the offline version be prepared?

### 6 Risks / Blockers

- API Key management is unclear
- On-site network may be unstable

# Skill 2：Requirements Breakdown Skill

## Role

Break meeting conclusions or vague ideas into structured requirements

## Applicable Scenarios

- product ideas
- business requirements
- feature discussions
- project initiation
- MVP planning

## Problem Solved

Without Skill when used

LLM may only give a suggestion:"you can first develop the functions for uploading materials and generating outlines and quiz questions."

With Skill With Skill

the output will always include 8 structured fields, forming a complete requirements document

With Skill standard output

- **Problem Statement**
- **User Stories**
- **Functional Requirements**
- **Non-functional RequirementsRequirements**
- **Acceptance Criteria**
- **Edge Cases**
- **Open Questions**
- **Implementation Phases**

# Requirements Breakdown Skill Output Effect

## Requirements Breakdown

Example output of a structured requirements document

### 1 Problem Statement

Employees need a simple AI-assisted training platform that can turn uploaded training materials into course outlines and quiz questions.

### 2 User Stories

- As an**employee**, I want to upload training materials so that I can generate learning content quickly
- As a**trainer**, I want the system to generate course outlines so that I can prepare sessions faster
- As a**tester**, I want generated quiz questions so that I can validate employee learning

### 3 Functional Requirements

- Users can upload training materials
- The system can generate course outlines
- The system can generate quiz questions
- The system supports an MVP version for initial rollout

### 4 Non-functional Requirements

- The system should support offline fallback
- The system should handle unstable network conditions
- API Key usage should be controlled

### 5 Acceptance Criteria

- **Given**a valid training document,**when**the user uploads it,**then**the system generates a course outline
- **Given**a course outline,**when**the user requests quiz questions,**then**the system generates at least 5 questions

# Test Case Generation Skill Output Effect

## Test Case 1: Upload Training Material

TC-001

**TEST SCENARIO**
User uploads a valid training document

**PRE-CONDITIONS**
User is logged in and has upload permission

**TEST STEPS**
1. Navigate to upload page
2. Select a PDF file (max 10MB)
3. Click upload button

**EXPECTED RESULT**
File is uploaded successfully, system confirms receipt

Priority: High | Test Type: Functional

## Test Case 2: Generate Course Outline

TC-002

**TEST SCENARIO**
System generates course outline from uploaded material

**PRE-CONDITIONS**
Training material is uploaded and processed

**TEST STEPS**
1. Select uploaded material
2. Click "Generate Outline" button
3. Wait for processing

**EXPECTED RESULT**
Course outline is generated with at least 3 sections

Priority: High | Test Type: Functional

## Test Case 3: Offline Fallback

TC-003

**TEST SCENARIO**
System works in offline mode

**PRE-CONDITIONS**
Network is disconnected

**TEST STEPS**
1. Disconnect network
2. Try to access previously uploaded materials
3. Verify cached content is available

**EXPECTED RESULT**
User can access cached content without network

Priority: Medium | Test Type: Non-functional

# Skill 4：Decision Memo Skill

## Organize options, risks, and decisions into a formal memo

- **project decision records**
- **risk assessment reports**
- **option approval**
- **management reporting**
- **compliance archiving**

Without SKILL
when used

LLM may only give vague advice:

> "You can record the decision and risks."

With SKILL
With Skill
the output always includes 8 fields:

| | |
|---|---|
| Decision Summary | Background |
| Options Considered | Decision Rationale |
| Risks and Mitigations | Action Items |
| Stakeholders | Approval Status |

**turn discussion outcomes into a formal decision document**

# Decision Memo Skill Output Effect

Decision Summary

**Proceed with MVP development of Employee AI Training Platform, targeting completion by next Wednesday**

## Background

The team discussed the need for an AI-assisted training platform to help employees learn faster. The initial version will focus on core functionality: uploading materials and generating outlines and quizzes.

## Options Considered

Rejected Build full platform immediately — due to time constraints

Selected Build MVP first — for faster delivery and validation

## Decision Rationale

- Faster time-to-market
- Lower initial investment
- Ability to gather user feedback early
- Reduced risk of over-engineering

## Risks and Mitigations

| Risk | Impact | Mitigation |
| --- | --- | --- |
| API Key management unclear | High | Define distribution process before launch |
| Network instability | Medium | Implement offline fallback mode |

## Action Items

**Zhang San** Complete backend API by next Wednesday

**Li Si** Complete frontend pages by next Wednesday

**Wang Wu** Prepare test cases by next Wednesday

Final effect: turn discussion outcomes into a formal decision document

# Harness Key Checkpoints

## Input Guardrail

- Does it contain sensitive information (passwords, keys, personal privacy)?
- Does the input format match expectations (text, JSONfiles)?
- Is the input length within a reasonable range?

## Intent Classification

- Is the intent clear?
- Is clarification needed?
- Does it match a known Skill
- Is multi- Skill collaboration needed?

## Output Validation

- Does the output format comply with the Skill definition?
- Are all required fields complete?
- Does the content comply with business rules?
- Does it include sensitive information leakage?

## Human Review Gate

- First run of a new Skill
- Output confidence below threshold
- Involves high-risk decisions
- User explicitly requests review

## Audit Logger

Input content snapshot | Output content snapshot | Skill call chain | Review decision record

Timestamp and operator

### Input Guardrail

- Sensitive information detection
- Format compliance validation
- Length range check
- Malicious instruction blocking

### Intent Classification

- Intent clarity assessment
- Clarification need detection
- Skill matching validation
- multi- Skill collaboration assessment

### Output Validation

- Format definition compliance
- Field completeness validation
- Business rule compliance
- Sensitive information leakage detection

### Human Review Gate

- New Skill first run
- Confidence below threshold
- High-risk decision scenario
- User proactively requests review

# From a Single Agent to an Organization-level Capability Marketplace

• When multiple Agents, Skills, tools, and knowledge bases appear inside an enterprise, the absence of a unified catalog leads to duplicated work, shadow Agents, unclear permissions, and missing evaluations.

• An organization-level capability marketplace makes Skills, Tools, Harness Templates, Agents, evaluation sets, and monitoring dashboards discoverable, requestable, reusable, approvable, and decommissionable.

• A capability marketplace is more than a “component store”; it also carries governance responsibilities: risk levels, data classification, owners, versions, dependencies, evaluation results, release status, and audit links.

• Banks need governable Agents more than more uncontrollable Agents. A platform-based capability ecosystem allows innovation without leaving compliance boundaries.


# Thanks

2026.7

ICSOFT Intelligent Computing and Software Center
浙江大学
智能计算与软件中心

www.icsoft.zju.edu.cn